

Phase 2 — RAG sur la documentation Python

Cinq concepts essentiels + six exercices pour aller plus loin

Cours PyTutor2

Résumé

Cette phase 2 construit, brique par brique, un système de *Retrieval-Augmented Generation* (RAG) qui répond à des questions en s'appuyant sur la documentation Python officielle. On y aborde : l'indexation (Web → embeddings → Chroma persistante), le `Retriever` LCEL, la chaîne RAG canonique en LCEL, la récupération des sources avec `RunnableParallel`, et la comparaison *similarity vs MMR*. Chaque exercice suit la même structure que la Phase 1 (énoncé + objectifs en boîte, solution détaillée, vérification LangSmith, récapitulatif). Une seconde partie « *Pour aller plus loin* » prolonge avec six exercices RAG avancés : impact du `chunk_size`, mode interactif CLI, prompt strict anti-hallucination, observabilité avec LangSmith, `MultiQueryRetriever`, et filtrage par métadonnée.

Table des matières

I Exercices	2
1 Indexation : du Web vers la base vectorielle	2
1.1 Solution	3
1.1.1 Architecture	3
1.1.2 Le code complet	3
1.1.3 Explications pas à pas	4
1.2 Vérification dans LangSmith	6
2 Retriever : recherche par similarité	7
2.1 Solution	7
2.1.1 Architecture	7
2.1.2 Le code complet	7
2.1.3 Explications pas à pas	8
2.2 Vérification dans LangSmith	9
3 Chaîne RAG simple en LCEL	10
3.1 Solution	10
3.1.1 Architecture	10
3.1.2 Le code complet	11
3.1.3 Explications pas à pas	12
3.2 Vérification dans LangSmith	13
4 RAG avec sources : récupérer réponse et contexte	15
4.1 Solution	15
4.1.1 Architecture	15
4.1.2 Le code complet	15
4.1.3 Explications pas à pas	16
4.2 Vérification dans LangSmith	18

5	Recherche <i>similarity</i> vs <i>MMR</i>	19
5.1	Solution	19
5.1.1	Architecture	19
5.1.2	Le code complet	19
5.1.3	Explications pas à pas	20
5.2	Vérification dans LangSmith	21
II	Pour aller plus loin	22
6	Impact du <code>chunk_size</code> sur la qualité du RAG	22
6.1	Solution	23
6.1.1	Architecture	23
6.1.2	Le code complet	23
6.1.3	Explications pas à pas	24
6.2	Vérification dans LangSmith	26
7	Boucle interactive : transformer le RAG en mini-CLI	26
7.1	Solution	27
7.1.1	Architecture	27
7.1.2	Le code complet	27
7.1.3	Explications pas à pas	28
7.2	Vérification dans LangSmith	30
8	Prompt strict pour refuser le hors-contexte	30
8.1	Solution	30
8.1.1	Architecture	30
8.1.2	Le code complet	31
8.1.3	Explications pas à pas	31
8.2	Vérification dans LangSmith	33
9	Vérifier et exploiter LangSmith	34
9.1	Solution	34
9.1.1	Architecture	34
9.1.2	Le code complet	34
9.1.3	Explications pas à pas	35
9.2	Vérification dans LangSmith	37
10	MultiQueryRetriever : reformuler la question	37
10.1	Solution	38
10.1.1	Architecture	38
10.1.2	Le code complet	38
10.1.3	Explications pas à pas	39
10.2	Vérification dans LangSmith	41
11	Filtrage par métadonnée	42
11.1	Solution	42
11.1.1	Architecture	42
11.1.2	Le code complet	43
11.1.3	Explications pas à pas	43
11.2	Vérification dans LangSmith	45
A	Programme principal exécutable	46

Première partie

Exercices

1 Indexation : du Web vers la base vectorielle

Exercice 1. Construire un index Chroma persistant

Aspire 7 pages de la documentation officielle Python (docs.python.org/3/tutorial/), découpe-les en *chunks*, encode-les en vecteurs avec un modèle d'embeddings local, et persiste l'index sur disque dans un dossier `chroma_db/`.

Tu écriras une fonction `build_or_load_vectorstore(force_rebuild=False)` qui : (a) **re-charge** l'index depuis le disque si `chroma_db/` existe ; (b) **reconstruit** l'index si l'argument `force_rebuild` est vrai ou si le dossier n'existe pas.

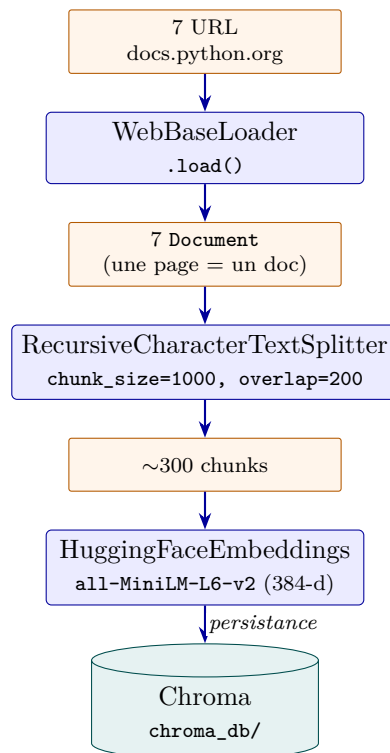
Contrainte : l'indexation initiale prend ~30 s (téléchargement des embeddings + scraping des URL) ; les lancements suivants doivent prendre <1 s grâce à la persistance.

Objectifs pédagogiques

- Utiliser `WebBaseLoader` pour aspirer une liste d'URL et obtenir des `Document` LangChain (texte + metadata).
- Choisir un `chunk_size` et un `chunk_overlap` cohérents pour de la doc technique, avec `RecursiveCharacterTextSplitter`.
- Encoder du texte en vecteurs avec `HuggingFaceEmbeddings` (modèle local, gratuit).
- Persister un `Chroma` sur disque et savoir le recharger sans réindexer.

1.1 Solution

1.1.1 Architecture



Un pipeline en quatre étapes : **scraping**, **découpage**, **encodage**, **stockage**. La séparation entre

indexation (lente, faite une fois) et interrogation (rapide, à chaque question) est essentielle pour un RAG utilisable au quotidien.

1.1.2 Le code complet

```
1 from pathlib import Path
2
3 from langchain_community.document_loaders import WebBaseLoader
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5 from langchain_huggingface import HuggingFaceEmbeddings
6 from langchain_chroma import Chroma
7
8
9 PYTHON_DOCS_URLS = [
10     "https://docs.python.org/3/tutorial/controlflow.html",
11     "https://docs.python.org/3/tutorial/datastructures.html",
12     "https://docs.python.org/3/tutorial/classes.html",
13     "https://docs.python.org/3/tutorial/errors.html",
14     "https://docs.python.org/3/tutorial/modules.html",
15     "https://docs.python.org/3/howto/functional.html",
16     "https://docs.python.org/3/glossary.html",
17 ]
18
19 PERSIST_DIR = "./chroma_db"
20 EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
21 COLLECTION_NAME = "python_docs"
22
23
24 def get_embeddings() -> HuggingFaceEmbeddings:
25     """Singleton de fait : un seul modèle d'embeddings réutilisé partout."""
26     return HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
27
28
29 def build_or_load_vectorstore(force_rebuild: bool = False) -> Chroma:
30     """Recharge l'index si chroma_db/ existe, sinon le construit."""
31     embeddings = get_embeddings()
32     persist_path = Path(PERSIST_DIR)
33
34     if persist_path.exists() and not force_rebuild:
35         print(f"V Rechargement de l'index depuis {PERSIST_DIR}")
36         return Chroma(
37             collection_name=COLLECTION_NAME,
38             persist_directory=PERSIST_DIR,
39             embedding_function=embeddings,
40         )
41
42     # --- 1. Scraping : URL → Document ---
43     loader = WebBaseLoader(PYTHON_DOCS_URLS)
44     loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}
45     raw_docs = loader.load()
46
47     # --- 2. Découpage : Document → chunks ---
48     splitter = RecursiveCharacterTextSplitter(
49         chunk_size=1000,
50         chunk_overlap=200,
51         separators=["\n\n", "\n", ". ", " ", ""],
52     )
53     chunks = splitter.split_documents(raw_docs)
54
```

```

55 # --- 3. Encodage + 4. Persistance (en une étape) ---
56 vectorstore = Chroma.from_documents(
57     documents=chunks,
58     embedding=embeddings,
59     collection_name=COLLECTION_NAME,
60     persist_directory=PERSIST_DIR,
61 )
62 return vectorstore

```

1.1.3 Explications pas à pas

Étape 1 — WebBaseLoader : du Web au Document

WebBaseLoader(urls) accepte une URL ou une liste d'URL. Chaque page devient un `langchain_core.documents`

```

1 Document(
2     page_content="Texte complet de la page...",
3     metadata={"source": "https://docs.python.org/3/tutorial/classes.html",
4               "title": "9. Classes"},
5 )

```

Le champ `metadata["source"]` est précieux : il permettra plus tard de **citer les sources** et de filtrer.

User-Agent personnalisé : `loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}.` Certains sites bloquent les bots anonymes ; mieux vaut s'identifier explicitement.

Étape 2 — RecursiveCharacterTextSplitter : le découpage

Un Document de 50 000 caractères est inutilisable directement : trop gros pour le contexte du LLM, et la recherche par similarité perd en précision sur des blocs trop larges. On découpe en *chunks* :

```

1 RecursiveCharacterTextSplitter(
2     chunk_size=1000,                                # en CARACTÈRES
3     chunk_overlap=200,                              # 20% de chevauchement
4     separators=["\n\n", "\n", ". ", " ", ""],      # ordre prioritaire
5 )

```

Trois paramètres clés :

- **chunk_size=1000** : en caractères, *pas en tokens*. 1000 caractères $\approx$ 200–250 tokens — bon compromis pour de la doc technique.
- **chunk_overlap=200** : 20% de chevauchement entre chunks consécutifs. Évite de couper une idée en deux.
- **separators** : l'ordre compte. Le splitter essaie d'abord `"\n\n"` (paragraphe), puis `"\n"` (ligne), puis `". "` (phrase), et en dernier recours au milieu d'un mot.

La taille du chunk est l'hyperparamètre n°1 de qualité d'un RAG. Trop petit : contexte fragmenté, le modèle manque le sens global. Trop grand : le retriever distingue mal les passages pertinents des passages tangentiels. 800-1200 caractères est le sweet spot pour de la doc technique.

Étape 3 — HuggingFaceEmbeddings : encoder en vecteurs

L'embedding transforme une chaîne de caractères en vecteur dense de dimension fixe (~384 ici). Deux passages sémantiquement proches donnent des vecteurs proches en distance cosinus.

```
1 HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

Le modèle all-MiniLM-L6-v2 :

- **384 dimensions** (compact, rapide)
- **Multilingue** (utile pour des questions en français sur de la doc anglaise)
- **Local** (pas d'appel API, pas de coût par requête)
- **~80 MB** (téléchargé une fois, mis en cache)

Étape 4 — Chroma : stocker et rechercher

Chroma est une base de données vectorielle qui :

- Indexe les vecteurs pour la recherche par similarité.
- Stocke `page_content` et `metadata` à côté.
- Persiste dans un dossier sur disque (`persist_directory`).

`Chroma.from_documents(...)` fait tout en une étape : calcule les embeddings, indexe, persiste. Pour recharger plus tard :

```
1 Chroma(  
2     collection_name="python_docs",  
3     persist_directory="./chroma_db",  
4     embedding_function=embeddings,  # même modèle qu'à l'indexation !  
5 )
```

Étape 5 — Le pattern *lazy rebuild*

L'indexation est lente. On la fait une fois et on recharge ensuite :

```
1 if persist_path.exists() and not force_rebuild:  
2     return Chroma(...)  # rechargement <1s  
3 # sinon, indexation complète
```

C'est le pattern à connaître pour tous les artefacts coûteux à construire : embeddings, modèles fine-tunés, caches de prompts, etc.

Sortie de l'exécution

```
-> Indexation de 7 pages de docs.python.org...  
    7 pages chargées  
    327 chunks générés  
[OK] Index créé et persisté dans ./chroma_db
```

1.2 Vérification dans LangSmith

L'indexation pure n'implique **pas** d'appel LLM, donc rien à voir dans LangSmith ici. Tu peux vérifier le contenu de l'index directement :

```
1 vs = build_or_load_vectorstore()  
2 print(vs._collection.count())  # nombre de chunks indexés
```

```

3 docs = vs.similarity_search("lambda", k=3)
4 for d in docs:
5     print(d.metadata["source"], d.page_content[:100])

```

*LangSmith ne trace que les **Runnable** (chaînes, appels LLM, retrievers). L'indexation Chroma se passe en dehors de ce périmètre — c'est pourquoi on l'isole soigneusement du reste du code.*

Récapitulatif

Ce qu'il faut retenir :

1. Le pipeline d'indexation est universel : *Loader* → *Splitter* → *Embeddings* → *VectorStore*.
2. *chunk_size* est le **premier** hyperparamètre à régler. 800–1200 caractères pour de la doc technique.
3. *chunk_overlap* ≈ 20% du *chunk_size* évite de couper une idée en deux.
4. *HuggingFaceEmbeddings* est local, gratuit, multilingue. L'alternative *OpenAIEmbeddings* est plus performante mais payante.
5. *Chroma* persiste sur disque. Pattern lazy rebuild : recharger si l'index existe, sinon reconstruire.

2 Retriever : recherche par similarité

Exercice 2. Transformer un vectorstore en Runnable LCEL

À partir d'un **Chroma** indexé, construis un **Retriever** qui prend une chaîne de caractères en entrée et renvoie les **k** documents les plus pertinents par similarité cosinus.

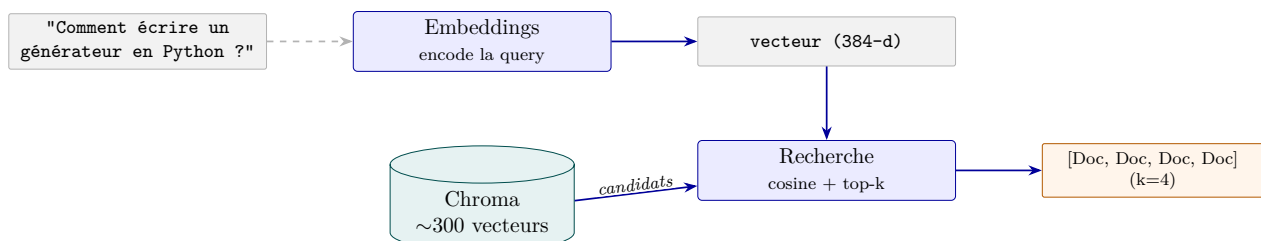
Contrainte : le **Retriever** doit être un **Runnable LCEL**, c'est-à-dire qu'il doit pouvoir être composé avec `|`, batché, streamé, et tracé dans LangSmith comme n'importe quel autre maillon.

Objectifs pédagogiques

- Comprendre comment `vectorstore.as_retriever()` produit un **Runnable** à partir d'un index **Chroma**.
- Paramétrer la recherche : **k** (nombre de docs retournés), **search_type** ("similarity" par défaut).
- Savoir lire ce qui se passe sous le capot : *embedding de la query* → *produit scalaire* → *tri* → *top-k*.

2.1 Solution

2.1.1 Architecture



Le retriever fait deux choses : (1) encoder la *query* avec le même modèle d'embeddings que les documents, (2) chercher les **k** vecteurs les plus proches par similarité cosinus.

2.1.2 Le code complet

```
1 # Le retriever est obtenu d'une seule ligne à partir du vectorstore.
2 vectorstore = build_or_load_vectorstore()
3
4 retriever = vectorstore.as_retriever(
5     search_type="similarity",      # défaut, on l'écrit pour la clarté
6     search_kwargs={"k": 4},        # nombre de docs renvoyés
7 )
8
9 # Le retriever est un Runnable : .invoke, .batch, .stream marchent
10 docs = retriever.invoke("Comment écrire un générateur en Python ?")
11
12 for i, d in enumerate(docs, 1):
13     print(f"[{i}] {d.metadata['source']}")
14     print(d.page_content[:200], "...")
```

2.1.3 Explications pas à pas

Étape 1 — .as_retriever() : du store au Runnable

Un Chroma (ou n'importe quel VectorStore LangChain) n'est pas directement un Runnable. La méthode .as_retriever() en crée un, qui peut alors être composé dans une chaîne LCEL avec l'opérateur |.

```
1 retriever = vectorstore.as_retriever(...)
2 # retriever est un Runnable
3 type(retriever).__mro__ # contient Runnable[str, list[Document]]
```

Le contrat de type : `str → list[Document]`.

Étape 2 — Ce qui se passe à l'.invoke()

1. La query (str) est encodée avec le **même HuggingFaceEmbeddings** qui a indexé les documents. C'est crucial : utiliser un autre modèle d'embeddings produirait un vecteur incomparable avec ceux de l'index.
2. Chroma calcule la similarité cosinus entre le vecteur query et **tous** les vecteurs indexés (en pratique avec un index approximatif pour aller vite).
3. Les k chunks de plus haute similarité sont renvoyés, triés par score décroissant.

Pourquoi cosinus et pas euclidienne ? La distance euclidienne dépend de la norme des vecteurs ; la similarité cosinus ne dépend que de l'angle. Pour des embeddings de phrases, c'est l'angle qui porte le sens.

Étape 3 — Le paramètre k

k est le nombre de chunks à renvoyer. C'est l'hyperparamètre n°2 de qualité du RAG (après chunk_size) :

- k=1 : trop pauvre, on rate souvent du contexte adjacent.
- k=4 à k=6 : standard pour de la doc technique.
- k=10+ : contexte saturé, le LLM peut perdre le fil. Coût en tokens augmenté.

Étape 4 — Le retriever dans une chaîne LCEL

Comme tout Runnable, le retriever se compose :

```
1 # Construire un mini-pipeline
2 def truncate(docs, n=3):
3     return docs[:n]
4
5 mini_chain = retriever | truncate
6 # Mini-chain reçoit une str et renvoie les 3 premiers docs
```

On verra dans l'exercice 3 comment l'enchaîner avec un prompt et un modèle pour faire un RAG complet.

Sortie de l'exécution

```
[1] https://docs.python.org/3/tutorial/modules.html
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'ZeroDivisionError', '_', '__build_class__',
'__debug__', '__doc__', '__import__', '__name__', '__pac ...
[2] https://docs.python.org/3/howto/functional.html
GeneratorExit
occurs, I suggest
using a
try:
...
finally:
suite instead of catching
GeneratorExit
.
The cumulative effect of these changes is to turn generators from one-way
producers of information i ...
[3] https://docs.python.org/3/tutorial/classes.html
9. Classes
9.1. A Word About Names and Objects
9.2. Python Scopes and Namespaces
9.2.1. Scopes and Namespaces Example
9.3. A First Look at Classes
9.3.1. Class Definition Syntax
9.3.2. Class Objects
9 ...
[4] https://docs.python.org/3/howto/functional.html
this case. Don't just use its value in expressions unless you're sure that
the send() method will be the only method used to resume your generator
function. In addition to send(), there are two othe ...
```

2.2 Vérification dans LangSmith

Quand le retriever est utilisé dans une chaîne tracée, LangSmith affiche :

```
Retriever (VectorStoreRetriever)
Input : "Comment écrire un générateur en Python ?"
Output : [Document, Document, Document, Document]

[0] source: tutorial/classes.html
[1] source: howto/functional.html
[2] ...
```

L'onglet Output montre la liste des *Document* renvoyés avec leur *metadata* (notamment

source). C'est le **premier endroit où regarder** quand un RAG répond mal : souvent le retriever a ramené du contexte hors-sujet.

Récapitulatif

Ce qu'il faut retenir :

1. `vectorstore.as_retriever(...)` transforme un store en *Runnable LCEL*, composable avec `|`.
2. Contrat du retriever : `str` $\rightarrow$ `list[Document]`.
3. Sous le capot : encode la query avec le même modèle d'embeddings, cherche les top-**k** par cosinus.
4. **k=4** à **k=6** est le standard pour de la doc technique.
5. Premier point de débogage d'un RAG cassé : regarder l'output du retriever dans LangSmith.

3 Chaîne RAG simple en LCEL

Exercice 3. Composer un RAG end-to-end en LCEL

Construis une chaîne *Runnable* qui prend une **question en texte brut** et renvoie une **réponse en texte brut**, en s'appuyant sur le retriever de l'exercice 2 pour récupérer du contexte pertinent dans la doc Python.

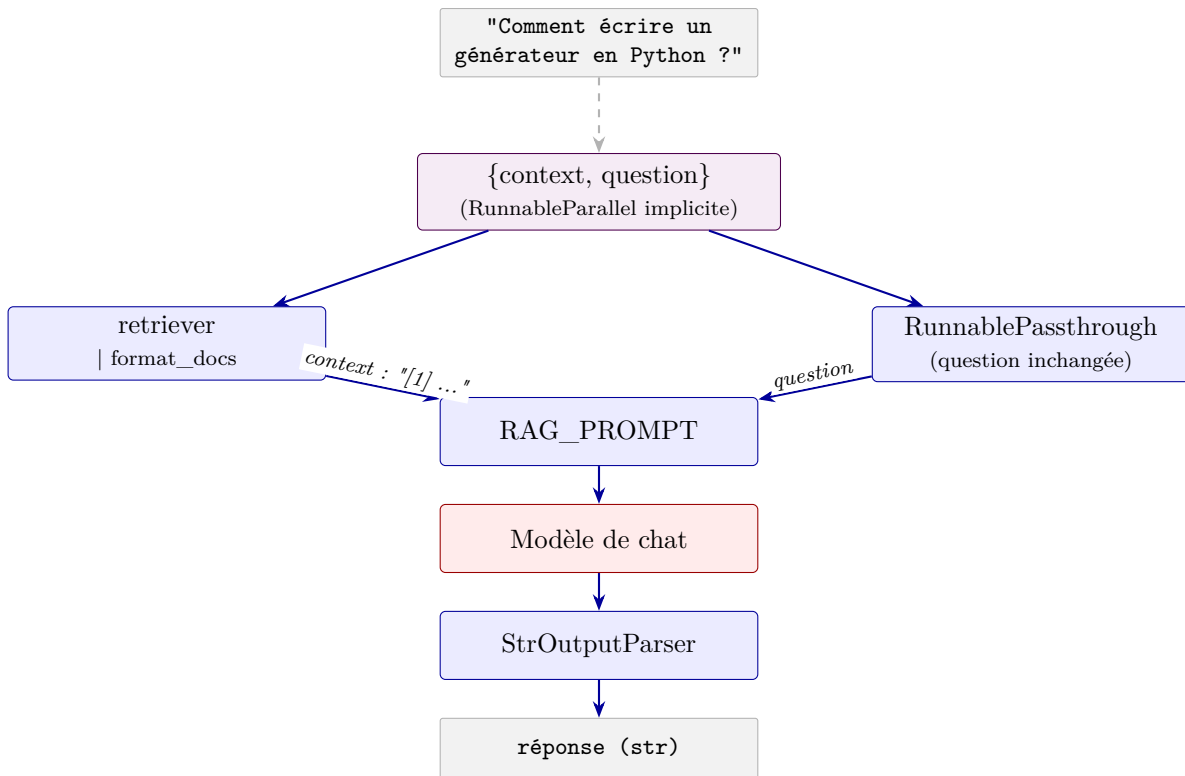
Contrainte : utiliser le pattern canonique LCEL avec un dict implicite qui contient deux branches `context` et `question`, puis composer `| prompt | model | StrOutputParser()`. Le modèle doit **citer ses sources** par numéro [N].

Objectifs pédagogiques

- Reconnaître le **pattern canonique RAG en LCEL** : `{context, question} | prompt | model | parser`.
- Utiliser `RunnablePassthrough()` pour faire passer la question à travers une branche du dict implicite.
- Composer `retriever | format_docs` pour transformer la liste de `Document` en string utilisable dans le prompt.
- Donner au modèle des instructions claires pour s'appuyer sur le contexte fourni et citer les sources.

3.1 Solution

3.1.1 Architecture



L'input (str) est dupliqué dans deux branches qui s'exécutent en parallèle : l'une va chercher le contexte, l'autre laisse passer la question. Les deux résultats sont assemblés dans un dict que le prompt sait formater.

3.1.2 Le code complet

```
1 from langchain_core.documents import Document
2 from langchain_core.prompts import ChatPromptTemplate
3 from langchain_core.output_parsers import StrOutputParser
4 from langchain_core.runnables import RunnablePassthrough
5 from config import get_model
6
7
8 RAG_PROMPT = ChatPromptTemplate.from_messages([
9     (
10         "system",
11         "Tu es un assistant pédagogique pour des élèves de Python. "
12         "Réponds à la question en t'appuyant UNIQUEMENT sur le contexte fourni.\n"
13         "Si la réponse n'est pas dans le contexte, dis-le explicitement plutôt "
14         "que d'inventer. Cite le ou les chunks utilisés par leur numéro entre "
15         "crochets, par exemple [1] ou [2,3].",
16     ),
17     (
18         "human",
19         "Contexte :\n{context}\n\n"
20         "Question : {question}\n\n"
21         "Réponse pédagogique :",
22     ),
23 ])
24
```

```

25
26 def format_docs(docs: list[Document]) -> str:
27     """Concatène les documents en un bloc de contexte numéroté."""
28     return "\n\n".join(
29         f"[{i + 1}] (source: {doc.metadata.get('source', '?')})\n{doc.page_content}"
30         for i, doc in enumerate(docs)
31     )
32
33
34 def build_rag_chain(retriever):
35     """Chaîne RAG simple : str -> str."""
36     return (
37         {
38             "context": retriever | format_docs,
39             "question": RunnablePassthrough(),
40         }
41         | RAG_PROMPT
42         | get_model()
43         | StrOutputParser()
44     )
45
46
47 # Usage
48 vectorstore = build_or_load_vectorstore()
49 retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
50 chain = build_rag_chain(retriever)
51
52 answer = chain.invoke("Comment écrire un générateur en Python ?")
53 print(answer)

```

3.1.3 Explications pas à pas

Étape 1 — Le dict implicite = RunnableParallel

```

1 {
2     "context": retriever | format_docs,
3     "question": RunnablePassthrough(),
4 }

```

Quand un dict apparaît dans une chaîne LCEL (au début ou via |), LCEL le convertit **implicitement** en `RunnableParallel`. Les deux branches reçoivent le même input (la question, str) et s'exécutent en parallèle :

- **branche context** : la string passe par `retriever` (qui renvoie une `list[Document]`), puis par `format_docs` (qui renvoie une str).
- **branche question** : la string traverse `RunnablePassthrough()` inchangée.

La sortie est un dict `{"context": "[1] ...", "question": "..."}` prêt à être consommé par le prompt.

Pattern à mémoriser : { "key1": branch1, "key2": branch2 } | prompt | model | parser. C'est la forme canonique du RAG en LCEL. Tu la reverras 1000 fois.

Étape 2 — retriever | format_docs

`retriever` produit une `list[Document]`. Le prompt attend une chaîne de caractères pour la variable `{context}`. La composition `retriever | format_docs` chaîne les deux :

```

1 def format_docs(docs):
2     return "\n\n".join(
3         f"[{i+1}] (source: {d.metadata['source']})\n{d.page_content}"
4         for i, d in enumerate(docs)
5     )

```

Trois détails :

- Numérotation [1], [2], ... : indispensable pour que le modèle puisse citer les sources *par numéro*.
- Ajout de la **source** dans chaque bloc : traçabilité, et permet au modèle de citer l'URL.
- Séparateur "\n\n" : les LLM repèrent mieux les blocs distincts ainsi qu'avec un simple "\n".

Étape 3 — RunnablePassthrough

`RunnablePassthrough()` est l'identité de LCEL : il prend un input et le renvoie inchangé. Utile dans un `RunnableParallel` pour *conserver* une valeur d'entrée tout en faisant autre chose avec elle dans une autre branche.

```

1 RunnablePassthrough().invoke("hello")    # → "hello"

```

Étape 4 — Le prompt RAG

Le prompt système contient **trois consignes** essentielles :

1. « UNIQUEMENT sur le contexte fourni » — anti-hallucination.
2. « Si la réponse n'est pas dans le contexte, dis-le » — préserve l'honnêteté épistémique.
3. « Cite les chunks par numéro [N] » — traçabilité.

Les trois consignes du prompt RAG sont défensives. Les LLM modernes ont tendance à compléter avec leurs connaissances générales — en RAG, c'est exactement ce qu'on veut éviter (sinon, à quoi sert le retrieval ?). On verra dans l'exercice bonus 3 comment durcir encore le prompt.

Étape 5 — StrOutputParser : extraire le texte

Le modèle renvoie un `AIMessage`. Pour récupérer juste le texte de la réponse, on ajoute `| StrOutputParser()` en fin de chaîne. C'est le même parseur qu'on utilisait en streaming dans la Phase 1.

Sortie de l'exécution

```

=====
DEMO 1 -- RAG simple (retriever similarity, k=4)
=====

Question : Comment écrire un générateur en Python ?

# Comment écrire un générateur en Python ?

D'après le contexte fourni, voici ce que je peux te dire sur les
générateurs :

## Définition
Un **générateur** est une fonction qui contient une ou plusieurs
expressions `yield` pour produire une série de valeurs [4].

## Caractéristiques principales
- Un générateur ressemble à une fonction normale, sauf qu'il utilise

```

```

    le mot-clé `yield` [4]
- Les valeurs produites peuvent être utilisées dans une boucle `for`
  ou récupérées une à une avec la fonction `next()` [4]

## Exemple de fonctionnement
Selon le contexte, quand tu appelles `next()` sur un générateur, il
exécute le code jusqu'à la prochaine expression `yield`. Quand il n'y
a plus de valeurs à produire, une exception `StopIteration` est
levée [1].

## Points importants à retenir
- Attention : `yield` retourne souvent `None`, donc tu dois vérifier
  cette valeur [1]
- Les générateurs ont des méthodes spéciales comme `throw()` et
  `close()` pour les contrôler [1]

**Cependant**, le contexte fourni ne contient pas d'exemple complet
de code montrant comment écrire un générateur pas à pas. Pour avoir
un exemple détaillé avec la syntaxe exacte, tu devrais consulter la
documentation Python officielle sur les générateurs.

```

3.2 Vérification dans LangSmith

L'arbre d'exécution attendu :

```

RunnableSequence
RunnableParallel<context, question>
[context] RunnableSequence
Retriever ← 1 query embedée
format_docs (RunnableLambda)
[question] RunnablePassthrough
ChatPromptTemplate
ChatAnthropic ← appel LLM
StrOutputParser

```

Les trois points à regarder :

1. **Output du Retriever** : les chunks ramenés sont-ils pertinents ? Si non, problème côté indexation ou k .
2. **Input du ChatAnthropic** : le prompt rendu, avec le contexte complet. C'est la *vérité terrain* de ce que le modèle a vu.
3. **Output du ChatAnthropic** : la réponse cite-t-elle bien les [N] ? Si non, le modèle a peut-être ignoré le contexte (à durcir avec le prompt strict de l'exercice bonus 3).

*Si la réponse est incorrecte, **ne blâme pas le modèle d'abord** : regarde l'output du retriever. 80% des bugs RAG viennent d'un mauvais retrieval (*chunk_size* mal réglé, *query* mal formulée, k trop petit).*

Récapitulatif

Ce qu'il faut retenir :

1. *Pattern canonique RAG : {context, question} / prompt / model / parser.*
2. Le **dict implicite** devient un `RunnableParallel` à deux branches : une qui va chercher (retriever / format), une qui passe la question.

3. *format_docs* numérote les chunks et expose les sources — condition pour que le modèle puisse citer.
4. Le prompt système doit explicitement : (1) restreindre au contexte, (2) autoriser le « je ne sais pas », (3) exiger les citations [N].
5. Pour debugger : lire d'abord l'output du retriever dans LangSmith, ensuite le prompt rendu.

4 RAG avec sources : récupérer réponse *et* contexte

Exercice 4. Préserver les documents sources dans la sortie

Modifie la chaîne RAG de l'exercice 3 pour qu'elle renvoie un **dict** `{"answer": str, "context": list[Document], "question": str}` au lieu de juste la réponse en string. Cela permet d'afficher les sources à l'utilisateur et de *vérifier* sur quels chunks la réponse s'est appuyée.

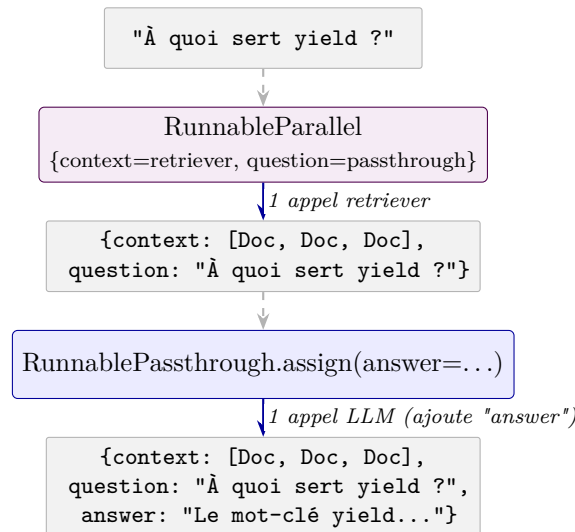
Contrainte : une seule invocation du retriever (pas deux). On utilise `RunnablePassthrough.assign` pour *enrichir* le dict au fil de la chaîne sans perdre les clés précédentes.

Objectifs pédagogiques

- Distinguer `RunnableParallel` (remplace l'input par un dict) de `RunnablePassthrough.assign` (enrichit le dict).
- Construire une chaîne en **deux phases** : récupération du contexte (1 appel retriever), puis génération de la réponse en gardant le contexte en mémoire.
- Comprendre le pattern *fan-out* / *accumulate* de LCEL.

4.1 Solution

4.1.1 Architecture



Deux étapes propres : d'abord on récupère le contexte une seule fois (branche `retrieve`), ensuite on ajoute la réponse au dict existant (`.assign`).

4.1.2 Le code complet

```

1 from langchain_core.runnables import RunnableParallel, RunnablePassthrough
2
3

```

```

4 def build_rag_chain_with_sources(retriever):
5     """Chaîne RAG qui renvoie un dict {answer, context, question}."""
6
7     # Étape 1 : récupère le contexte UNE FOIS et le passe en aval
8     retrieve = RunnableParallel(
9         context=retriever,
10        question=RunnablePassthrough(),
11    )
12
13    # Étape 2 : à partir de {context: [Doc...], question: str},
14    # construit la réponse en formatant les docs au passage.
15    answer_chain = (
16        RunnablePassthrough.assign(
17            context=lambda x: format_docs(x["context"]),
18        )
19        | RAG_PROMPT
20        | get_model()
21        | StrOutputParser()
22    )
23
24    # .assign(answer=...) ajoute la clé "answer" sans toucher aux autres.
25    return retrieve | RunnablePassthrough.assign(answer=answer_chain)
26
27
28 # Usage
29 chain = build_rag_chain_with_sources(retriever)
30 result = chain.invoke("À quoi sert le mot-clé yield exactement ?")
31
32 print("RÉPONSE :")
33 print(result["answer"])
34
35 print("\nSOURCES :")
36 for i, doc in enumerate(result["context"], 1):
37     print(f"  [{i}] {doc.metadata['source']}")
38     print(f"      {doc.page_content[:120]}...")

```

4.1.3 Explications pas à pas

Étape 1 — Récupérer une seule fois

```

1 retrieve = RunnableParallel(
2     context=retriever,
3     question=RunnablePassthrough(),
4 )

```

À partir d'une question (str), ce `RunnableParallel` produit :

- `context` : la `list[Document]` renvoyée par le retriever (**1 appel**).
- `question` : la string d'origine (passthrough).

Important : on ne formate *pas encore* les docs ici. On veut les garder bruts pour pouvoir les retourner à l'utilisateur en sortie.

Étape 2 — La sous-chaîne qui calcule la réponse

```

1 answer_chain = (
2     RunnablePassthrough.assign(

```



```

3     context=lambda x: format_docs(x["context"]),
4 )
5 | RAG_PROMPT | get_model() | StrOutputParser()
6 )

```

Cette sous-chaîne reçoit `{context: [Doc...], question: str}` et fait trois choses :

1. **Remplace** la clé `context` par sa version formatée en string. Le `RunnablePassthrough.assign(context=...)` applique la lambda à l'input courant et place le résultat dans `context`. La clé `question` est intacte.
2. Le prompt formate avec les variables `{context}` (str maintenant) et `{question}`.
3. Modèle + parser produisent la réponse en string.

`RunnablePassthrough.assign(key=fn)` remplace la clé `key` si elle existe déjà, ou la crée si elle n'existe pas. C'est l'équivalent d'un `UPDATE` en `SQL` sur un dict.

Étape 3 — L'accumulation finale

```

1 return retrieve | RunnablePassthrough.assign(answer=answer_chain)

```

Le `RunnablePassthrough.assign(answer=answer_chain)` prend le dict produit par `retrieve` (`{context: [Doc...], question: str}`) et y ajoute la clé `answer` (résultat de `answer_chain`).

La sortie finale :

```

1 {
2     "context": [Document, Document, Document, Document], # bruts, intacts
3     "question": "À quoi sert yield ?",                    # str
4     "answer": "Le mot-clé yield...",                     # str
5 }

```

L'utilisateur peut afficher `result["answer"]` et énumérer `result["context"]` pour montrer les sources.

Étape 4 — Pourquoi pas deux retrievers ?

Une approche naïve appellerait deux fois le retriever : une fois pour afficher les sources, une fois inclus dans la chaîne RAG. C'est deux fois plus de calcul et l'index pourrait même retourner des docs différents (selon la non-déterminisme).

Le pattern `RunnableParallel + .assign` :

- **1 seul appel** retriever (économie).
- **Cohérence** : la réponse s'appuie *exactement* sur les sources affichées (pas de divergence possible).
- **Composable** : tout reste un `Runnable`.

Sortie de l'exécution

```

=====
DEMO 2 -- RAG avec inspection des sources
=====

Question : A quoi sert le mot-clé `yield` exactement ?

REPONSE :
# A quoi sert le mot-clé `yield` ?

```

Le mot-clé ``yield`` sert à **créer des générateurs** en Python. Voici ses principales fonctions [1,2,3] :

1. Suspendre l'exécution et retourner une valeur
Quand ``yield`` est rencontré, la fonction s'arrête et retourne une valeur. A chaque appel suivant, l'exécution reprend à partir du ``yield`` [2].

2. Préserver l'état local
Contrairement à ``return`` qui termine la fonction, ``yield`` **suspend** l'exécution et préserve les variables locales. Cela permet à la fonction de reprendre son travail plus tard [2].

3. Recevoir des valeurs
``yield`` peut aussi **recevoir des valeurs** via la méthode ``send(value)``. La valeur envoyée devient le résultat de l'expression ``yield`` [1,3].

Exemple simple [3] :

```
```python
def counter(maximum):
 i = 0
 while i < maximum:
 val = (yield i) # Retourne i, puis attend une valeur
 if val is not None:
 i = val # Reçoit une nouvelle valeur
 else:
 i += 1
...
```
```

En résumé : ``yield`` transforme une fonction ordinaire en **générateur** qui produit une séquence de valeurs une à une, tout en gardant son état entre chaque appel. C'est très utile pour économiser la mémoire avec de grandes séquences [2].

SOURCES UTILISEES :

- [1] <https://docs.python.org/3/howto/functional.html>
are messy. In Python 2.5 there's a simple way to pass values into a generator. `yield` became an expression, returning a v...
- [2] <https://docs.python.org/3/howto/functional.html>
yield and a return statement is that on reaching a yield the generator's state of execution is suspended and local varia...
- [3] <https://docs.python.org/3/howto/functional.html>
+ 12 .) Values are sent into a generator by calling its `send(value)` method. This method resumes the generator's code an...
- [4] <https://docs.python.org/3/howto/functional.html>
GeneratorExit occurs, I suggest using a try: ... finally: suite instead of catching GeneratorExit . The cumulative effec...

4.2 Vérification dans LangSmith

L'arbre d'exécution est nettement plus riche :

```

RunnableSequence
RunnableParallel<context, question>
    [context] Retriever
    [question] RunnablePassthrough
    RunnableAssign<answer>
RunnableAssign<context=format_docs>
    ChatPromptTemplate
    ChatAnthropic ← appel LLM
    StrOutputParser

```

Dans l'onglet Output du run racine, tu vois directement le dict final avec ses 3 clés. C'est la matérialisation du pattern fan-out / accumule que l'on cherche en LCEL pour produire des sorties riches sans dupliquer les calculs.

Récapitulatif

Ce qu'il faut retenir :

1. `RunnableParallel` crée un dict; `Passthrough.assign` enrichit un dict existant. Ne pas confondre.
2. Pattern fan-out / accumule : une étape construit un dict, les étapes suivantes ajoutent des clés sans détruire les précédentes.
3. Une seule invocation du retriever — la réponse et les sources affichées sont cohérentes par construction.
4. Pour faire un RAG « professionnel » avec sources, c'est le pattern à connaître.

5 Recherche *similarity* vs *MMR*

Exercice 5. Comparer deux stratégies de retrieval

Compare deux `Retriever` sur la même question : l'un avec `search_type="similarity"` (par défaut), l'autre avec `search_type="mmr"` (*Maximal Marginal Relevance*). Observe que MMR produit des chunks plus *diversifiés* pour une question généraliste comme « *liste vs tuple* ».

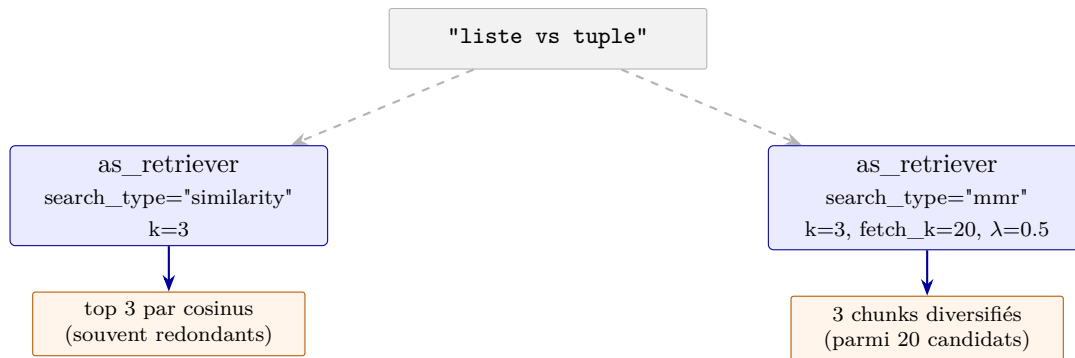
Contrainte : utiliser le même `Chroma`, juste changer `search_kwargs`. Pour MMR, expérimenter avec `fetch_k` (taille de la pool de candidats) et `lambda_mult` (équilibre similarité/diversité).

Objectifs pédagogiques

- Comprendre le problème de la *redondance sémantique* en retrieval pur par similarité.
- Connaître la *Maximal Marginal Relevance* (MMR) et ses deux paramètres : `fetch_k` et `lambda_mult`.
- Savoir choisir entre les deux stratégies selon la nature de la question (précise vs généraliste).

5.1 Solution

5.1.1 Architecture



5.1.2 Le code complet

```
1 vectorstore = build_or_load_vectorstore()
2 question = "Quelle est la différence entre une liste et un tuple ?"
3
4 # --- Retriever similarity (défaut) ---
5 sim_retriever = vectorstore.as_retriever(
6     search_type="similarity",
7     search_kwargs={"k": 3},
8 )
9
10 # --- Retriever MMR ---
11 mmr_retriever = vectorstore.as_retriever(
12     search_type="mmr",
13     search_kwargs={
14         "k": 3, # nombre final de docs renvoyés
15         "fetch_k": 20, # pool de candidats avant sélection MMR
16         "lambda_mult": 0.5, # 1.0 = pure similarité, 0.0 = pure diversité
17     },
18 )
19
20 # --- Comparaison ---
21 print("--- Top-3 similarity ---")
22 for i, doc in enumerate(sim_retriever.invoke(question), 1):
23     print(f" [{i}] {doc.page_content[:100]}...")
24
25 print("\n--- Top-3 MMR (diversifié) ---")
26 for i, doc in enumerate(mmr_retriever.invoke(question), 1):
27     print(f" [{i}] {doc.page_content[:100]}...")
```

5.1.3 Explications pas à pas

Étape 1 — Le problème de la similarité pure

La similarité cosinus classe les chunks par **proximité avec la query**, indépendamment les uns des autres. Conséquence : si la query matche fortement un certain passage, les **k** chunks les plus similaires sont souvent **très proches entre eux** :

Pour la query « liste vs tuple », les top-3 par similarité sont typiquement 3 paragraphes de la même section qui parlent de la même définition formelle. On rate les autres angles du sujet (mutabilité, performances, conventions d'usage).

Étape 2 — La *Maximal Marginal Relevance*

MMR sélectionne itérativement les chunks en équilibrant deux critères :

- **Pertinence** : similarité avec la query.
- **Diversité** : *dissimilarité* avec les chunks déjà sélectionnés.

Algorithme :

1. Récupérer les `fetch_k` candidats par similarité pure (ex : 20).
2. Sélectionner le candidat le plus proche de la query (top 1).
3. Pour chaque candidat restant, calculer :

$$\text{score} = \lambda \cdot \text{sim}(c, q) - (1 - \lambda) \cdot \max_{s \in S} \text{sim}(c, s)$$

où S est l'ensemble des candidats déjà sélectionnés.

4. Sélectionner le candidat de plus haut score. Répéter jusqu'à k .

Étape 3 — Les paramètres `fetch_k` et `lambda_mult`

| Paramètre | Effet |
|--------------------------|--|
| <code>k</code> | Nombre final de docs renvoyés. |
| <code>fetch_k</code> | Taille de la pool de candidats sur laquelle MMR opère. Plus c'est grand, plus la diversité <i>potentielle</i> est large. Typique : <code>fetch_k = 5 × k</code> . |
| <code>lambda_mult</code> | Curseur similarité ↔ diversité : <code>1.0</code> = MMR redevient similarité pure, <code>0.0</code> = pure diversité (déconnectée de la query). Sweet spot : <code>0.5</code> à <code>0.7</code> . |

Étape 4 — Quand utiliser MMR

| similarity (défaut) | mmr |
|---|---|
| Question précise, monolithique | Question généraliste, multifacette |
| « Comment fonctionne <code>super()</code> ? » | « C'est quoi la POO en Python ? » |
| On veut le <i>meilleur</i> match exact | On veut un <i>tour d'horizon</i> |
| Plus rapide (pas de recalcul) | Plus lent ($O(k \times \text{fetch_k})$) |

Règle pratique : si tes top-3 par similarité semblent redondants (3 passages disant la même chose), passe en MMR. Si les top-3 sont déjà variés, similarité suffit (et c'est plus rapide).

Sortie de l'exécution

```
=====
DEMO 3 -- Similarity vs MMR sur la MEME question
=====

Question : Quelle est la différence entre une liste et un tuple ?

--- Top-3 similarity ---
[1] , line 1 , in <module> TypeError : 'tuple' object does not
    support item assignment >>> # but they ca...
[2] >>> list ( x for x in range ( 10 ) if is_even ( x )) [0, 2, 4,
    6, 8] enumerate(iter, start=0) counts...
[3] elements that are accessed via unpacking (see later in this
```

```

        section) or indexing (or even by attribu...

--- Top-3 MMR (diversifié) ---
[1] , line 1 , in <module> TypeError : 'tuple' object does not
    support item assignment >>> # but they ca...
[2] 5.1. More on Lists The list data type has some more methods.
    Here are all of the methods of list o...
[3] sequences of the same type, the lexicographical comparison is
    carried out recursively. If all items...

=====
Observe : MMR évite les doublons sémantiques que retourne la
similarity pure. Pour une question généraliste comme 'liste vs
tuple', MMR ramène souvent des chunks plus complémentaires.
=====

```

5.2 Vérification dans LangSmith

Pour les deux retrievers, LangSmith affiche le même type de run (`VectorStoreRetriever`). La différence se voit dans le *contenu* des chunks renvoyés : tu compares visuellement les deux runs côte-à-côte dans l'UI.

LangSmith n'expose pas (encore) les scores MMR internes. Si tu veux les inspecter, utilise `vectorstore.max_marginal_relevance_search_with_score(...)` directement, en dehors de la chaîne.

Récapitulatif

Ce qu'il faut retenir :

1. La similarité pure peut être **redondante** : les top-*k* sont souvent des paragraphes proches du même passage.
2. MMR équilibre **pertinence et diversité** : pioche dans une pool de `fetch_k` candidats les *k* les plus complémentaires.
3. `lambda_mult` pilote le curseur : 0.5–0.7 est le sweet spot pour de la doc technique.
4. Bascule en MMR pour les questions **généralistes ou ouvertes** ; reste en *similarity* pour les questions précises.
5. Côté API : c'est juste un paramètre `search_type` à changer dans `.as_retriever(...)`.

Deuxième partie

Pour aller plus loin

6 Impact du `chunk_size` sur la qualité du RAG

Exercice bonus 1. Mesurer expérimentalement l'effet du découpage

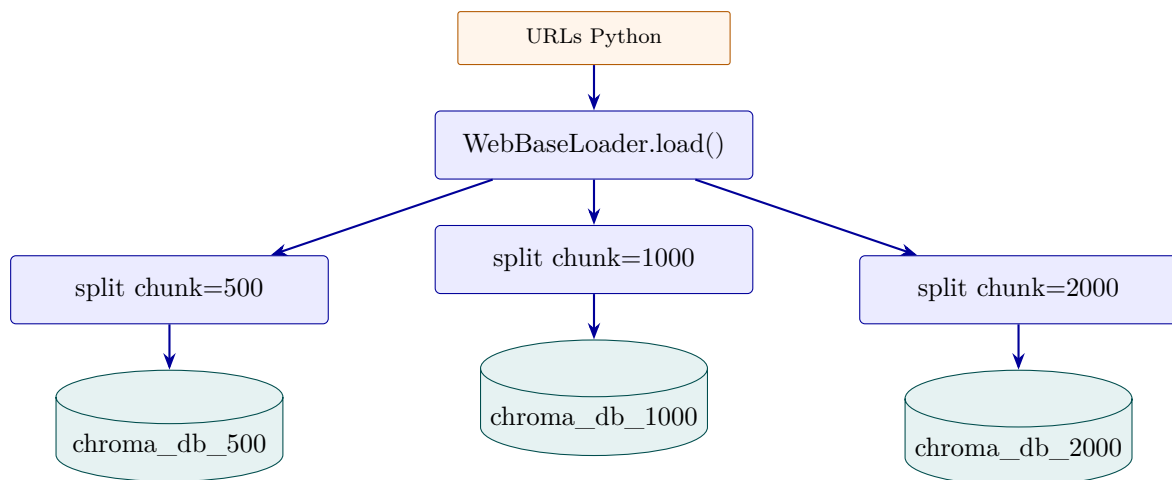
Construis trois vectorstores avec `chunk_size = 500, 1000, 2000` caractères (et `chunk_overlap` proportionnel, 20%). Pose la même question précise aux trois et compare (a) la longueur moyenne des chunks ramenés, (b) leur pertinence visuelle, (c) la qualité de la réponse finale. **Contrainte** : persister les trois index dans des dossiers séparés (`chroma_db_500`, etc.) pour éviter de réindexer à chaque test.

Objectifs pédagogiques

- Voir **concrètement** pourquoi le `chunk_size` est l'hyperparamètre n°1 de qualité RAG.
- Identifier le compromis *granularité vs contexte* : petit chunk = retrieval précis mais contexte fragmenté ; gros chunk = contexte riche mais retrieveur moins discriminant.
- Trouver expérimentalement le sweet spot pour un corpus donné.

6.1 Solution

6.1.1 Architecture



Trois index isolés, même corpus, paramètre de découpage différent. C'est le protocole expérimental le plus simple pour évaluer un hyperparamètre en RAG : *ceteris paribus*, on change une seule variable.

6.1.2 Le code complet

```
1 from pathlib import Path
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 from langchain_chroma import Chroma
4 from langchain_community.document_loaders import WebBaseLoader
5
6 from phase2_rag import (
7     PYTHON_DOCS_URLS, COLLECTION_NAME, get_embeddings, build_rag_chain,
8 )
9
```

```

10
11 def build_vectorstore_with_chunk_size(chunk_size: int, persist_dir: str) -> Chroma:
12     """Construit (ou recharge) un vectorstore avec un chunk_size donné."""
13     embeddings = get_embeddings()
14     if Path(persist_dir).exists():
15         return Chroma(
16             collection_name=COLLECTION_NAME,
17             persist_directory=persist_dir,
18             embedding_function=embeddings,
19         )
20
21     loader = WebBaseLoader(PYTHON_DOCS_URLS)
22     loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}
23     raw_docs = loader.load()
24
25     splitter = RecursiveCharacterTextSplitter(
26         chunk_size=chunk_size,
27         chunk_overlap=int(chunk_size * 0.2),          # 20% proportionnel
28         separators=["\n\n", "\n", ". ", " ", ""],
29     )
30     chunks = splitter.split_documents(raw_docs)
31
32     return Chroma.from_documents(
33         documents=chunks, embedding=embeddings,
34         collection_name=COLLECTION_NAME, persist_directory=persist_dir,
35     )
36
37
38 def exercise_1() -> None:
39     configs = [
40         (500, "./chroma_db_500"),
41         (1000, "./chroma_db_1000"),
42         (2000, "./chroma_db_2000"),
43     ]
44     question = "Comment lève-t-on une exception personnalisée en Python ?"
45
46     for chunk_size, persist_dir in configs:
47         print(f"\n--- chunk_size = {chunk_size} ---")
48         vs = build_vectorstore_with_chunk_size(chunk_size, persist_dir)
49         retriever = vs.as_retriever(search_kwargs={"k": 3})
50
51         docs = retriever.invoke(question)
52         avg_len = sum(len(d.page_content) for d in docs) // len(docs)
53         print(f" Longueur moyenne des chunks : {avg_len} caractères")
54
55         chain = build_rag_chain(retriever)
56         answer = chain.invoke(question)
57         print(f" Réponse : {answer[:300]}...")

```


6.1.3 Explications pas à pas

Étape 1 — Le compromis granularité vs contexte

| chunk_size | Avantages | Inconvénients |
|---------------|--|--|
| 500 | Recherche précise (chaque chunk est ciblé) | Contexte fragmenté, le modèle manque l'environnement de l'idée |
| 1000 (défaut) | Bon compromis pour de la doc technique | — |
| 2000 | Contexte riche, chunks auto-suffisants | Beaucoup de bruit, le retriever distingue mal pertinent / tangentiel |

Étape 2 — Mesurer plutôt que deviner

L'effet du `chunk_size` dépend du corpus. Sur la doc Python (passages bien structurés, paragraphes courts), 1000 est optimal. Sur des transcripts de conférences (idées qui s'étalent sur plusieurs paragraphes), 1500-2000 peut être meilleur. **Mesure** pour ton corpus.

*Le bon outil de mesure n'est pas la longueur ni le nombre de chunks : c'est la **qualité subjective des réponses** sur un échantillon de questions représentatives. Construis un mini-jeu de 10 questions typiques et compare les réponses.*

Étape 3 — Le `chunk_overlap` proportionnel

On garde `chunk_overlap = 0.2 × chunk_size` : 20% de chevauchement entre chunks consécutifs. C'est une bonne valeur *par défaut* qui évite de couper une idée en deux sans gonfler excessivement la taille de l'index.

Étape 4 — Persister chaque variante

Réindexer prend du temps. En persistant dans des dossiers séparés (`chroma_db_500`, etc.), on peut :

- Comparer les variantes rapidement (rechargement <1 s chacun).
- Garder un environnement reproductible.
- Faire le test une fois, l'oublier.

Sortie de l'exécution

```
=====
EXERCICE 1 -- Impact du chunk_size sur la qualité du RAG
=====

--- chunk_size = 500 ---
-> Indexation chunk_size=500 dans ./chroma_db_500...
-> 658 chunks générés
Longueur moyenne des chunks retrouvés : 472 caractères
[1] ) # arguments stored in .args ... print ( inst ) # __str__
      allows args to be pri...
[2] >>> try : ... raise TypeError ( 'bad type' ) ... except
      Exception as e : ... e ....
[3] know where to look in case the input came from a file. 8.2.
      Exceptions Even if...

Réponse :
# Lever une exception personnalisée en Python

D'après le contexte fourni, je peux te montrer comment lever une
```

```

exception en utilisant le mot-clé `raise` [2].

## Syntaxe de base

```python
raise TypeError('message')
```

...

--- chunk_size = 1000 ---
-> Indexation chunk_size=1000 dans ./chroma_db_1000...
-> 327 chunks générés
Longueur moyenne des chunks retrouvés : 981 caractères
[1] except Exception as inst : ... print ( type ( inst )) # the
    exception type ... p...
[2] They include SystemExit which is raised by sys.exit() and
    KeyboardInterrupt whic...
[3] err =} , { type ( err ) =} " ) raise The try ... except
    statement has an optional ...

Réponse :
Je dois être honnête : **la réponse à ta question n'est pas
présente dans le contexte fourni.**

Le contexte [1], [2] et [3] explique comment :
- **Capturer** des exceptions avec `try...except`
- **Accéder** aux informations d'une exception
- **Re-lever** une exception avec...

--- chunk_size = 2000 ---
-> Indexation chunk_size=2000 dans ./chroma_db_2000...
-> 165 chunks générés
Longueur moyenne des chunks retrouvés : 1977 caractères
[1] ( inst . args ) # arguments stored in .args ... print ( inst )
    # __str__ allows ...
[2] # shorthand for 'raise ValueError()' If you need to determine
    whether an excepti...
[3] +-+----- 1 ----- | OSError: error 1
    +----- 2 --...

Réponse :
# Comment lever une exception personnalisée en Python ?

D'après le contexte fourni, je peux te montrer comment lever une
exception en Python, mais la documentation ne couvre pas
explicitement la création d'exceptions personnalisées.
...

-----
OBSERVATIONS A RETENIR :
- chunk=500 : chunks précis mais souvent incomplets, le modèle
  peut manquer le contexte autour du détail recherché.
- chunk=1000 : le bon défaut pour de la doc technique.
- chunk=2000 : beaucoup de bruit, le retriever distingue mal
  les passages pertinents des passages tangentiels.

```

6.2 Vérification dans LangSmith

Pour chaque `chunk_size`, lance la même question et compare les runs côte-à-côte. Trois choses se révèlent :

1. **Tokens du prompt** : `chunk_size=500` avec `k=3` donne ~ 1500 tokens de contexte ; `chunk_size=2000` donne ~ 6000 .

2. **Pertinence des chunks** : lisible dans l'output du retrieveur.
3. **Qualité de la réponse** : peut être annotée manuellement dans LangSmith (*thumbs up/down*) pour mémoriser l'évaluation.

Le coût en tokens monte vite avec le `chunk_size` : à $k=3$, passer de 1000 à 2000 caractères double le coût par requête sans nécessairement améliorer la qualité. Optimise.

Récapitulatif

Ce qu'il faut retenir :

1. Le `chunk_size` est l'hyperparamètre n°1 de qualité RAG. Mesure, ne devine pas.
2. Sweet spot général pour de la doc technique : **800–1200 caractères**, `overlap = 20%`.
3. Petit chunk : précis mais fragmenté. Gros chunk : riche mais bruité.
4. Persiste chaque variante dans son dossier pour comparer sans réindexer.
5. Le coût en tokens augmente linéairement avec `chunk_size` $\times$ k .

7 Boucle interactive : transformer le RAG en mini-CLI

Exercice bonus 2. Une boucle while pour explorer la doc librement

Transforme la chaîne RAG en une petite REPL (*Read-Eval-Print Loop*) : une boucle `while` qui demande une question à l'utilisateur via `input()`, appelle la chaîne, affiche la réponse **et les sources**, puis recommence. Termine proprement sur ligne vide ou `Ctrl+C`.

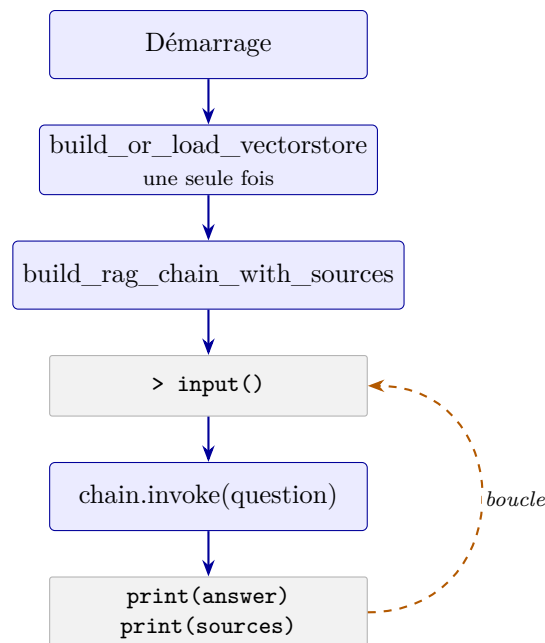
Contrainte : utiliser `build_rag_chain_with_sources` de l'exercice 4 pour afficher les sources, et gérer les exceptions `EOFError` / `KeyboardInterrupt` pour une sortie propre.

Objectifs pédagogiques

- Construire une mini-CLI qui sert de banc de test pour explorer son corpus librement.
- Gérer la terminaison propre (signal et ligne vide).
- Préparer mentalement la transition vers une UI web (Gradio, Streamlit) où la boucle d'événements est cachée par le framework.

7.1 Solution

7.1.1 Architecture



Trois éléments importants pour la performance perçue : charger le store **une seule fois** (lent), construire la chaîne **une seule fois**, et la réutiliser en boucle pour chaque question (rapide).

7.1.2 Le code complet

```
1 from phase2_rag import build_or_load_vectorstore, build_rag_chain_with_sources
2
3
4 def exercise_2_interactive() -> None:
5     print("PyTutor - mode interactif (Ctrl+C ou ligne vide pour quitter)")
6
7     # On charge le store et on construit la chaîne UNE SEULE FOIS
8     vectorstore = build_or_load_vectorstore()
9     retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
10    chain = build_rag_chain_with_sources(retriever)
11
12    while True:
13        try:
14            question = input("\n> ").strip()
15        except (EOFError, KeyboardInterrupt):
16            print("\nÀ la prochaine !")
17            break
18
19        if not question:
20            print("Au revoir.")
21            break
22
23        result = chain.invoke(question)
24
25        print("\n" + "-" * 70)
26        print(result["answer"])
27        print("-" * 70)
28        print("Sources :")
29        for i, doc in enumerate(result["context"], 1):
```

```
30     src = doc.metadata.get("source", "?").split("/")[-1]
31     print(f"    [{i}] {src}")
```

7.1.3 Explications pas à pas

Étape 1 — Init coûteux, hors de la boucle

```
1 vectorstore = build_or_load_vectorstore()           # 1s (rechargement)
2 retriever   = vectorstore.as_retriever(...)         # instantané
3 chain       = build_rag_chain_with_sources(...)     # instantané
4 # Boucle après l'init :
5 while True:
6     ...
```

Pattern universel des REPL : tout ce qui est lent (chargement, parsing, initialisation) se fait **une fois** avant la boucle. Dans la boucle, chaque tour ne fait que ce qui dépend de l'input.

Étape 2 — Gérer les sorties propres

Trois façons pour l'utilisateur de quitter :

- **Ctrl+C** : `input()` lève `KeyboardInterrupt`.
- **Ctrl+D** (EOF) : `input()` lève `EOFError`.
- **Ligne vide** : `input()` renvoie "", qu'on traite explicitement.

Le `try/except` attrape les deux premiers, le `if not question` attrape le troisième.

Sans `try/except`, un `Ctrl+C` propage une trace Python disgracieuse à l'utilisateur. L'attraper et imprimer un message amical (« À la prochaine ! ») est la marque du logiciel poli.

Étape 3 — Affichage avec sources

`build_rag_chain_with_sources` renvoie un dict avec `answer`, `context`, `question`. On affiche :

- La réponse, encadrée d'une ligne pour la séparer visuellement du prompt.
- Les sources, chacune avec son fichier (`.split("/")[-1]` pour ne garder que le *basename*).

Étape 4 — Vers une UI web

Cette mini-CLI est la base mentale d'une UI :

- **Gradio** :

```
1 import gradio as gr
2 gr.Interface(
3     fn=lambda q: chain.invoke(q)["answer"],
4     inputs="text", outputs="text",
5 ).launch()
```

- **Streamlit** : `st.text_input`, `st.write`.
- **FastAPI** : un endpoint `POST /ask` qui retourne le dict en JSON.

Dans tous les cas, la chaîne RAG reste un `Runnable` inchangé : on remplace juste l'interface utilisateur autour.

Sortie de l'exécution

```
=====
EXERCICE 2 -- Mode interactif (Ctrl+C ou ligne vide pour quitter)
=====

[OK] Rechargement de l'index depuis ./chroma_db

PyTutor est prêt. Pose une question sur Python.

> c'est quoi les fonctions décorées?

-----
# Les fonctions décorées

Une fonction décorée est une fonction qui a été transformée par
un décorateur. [1]

## Qu'est-ce qu'un décorateur ?

Un décorateur est une fonction qui retourne une autre fonction. Il
est généralement appliqué comme une transformation de fonction en
utilisant la syntaxe @nom_du_decorateur. [1]

## Exemple simple

Voici deux façons équivalentes de décorer une fonction : [1]

Syntaxe avec @` (sucre syntaxique):

```
python
@staticmethod
def f(arg):
 ...
```

Syntaxe sans @` (équivalent):

```
python
def f(arg):
 ...
f = staticmethod(f)
...
```



Les deux approches font exactement la même chose : elles appliquent
le décorateur staticmethod` à la fonction f`.

## Exemples courants

Les décorateurs les plus courants en Python sont : [1]
- @classmethod`
- @staticmethod`

Ces décorateurs modifient le comportement de la fonction en la
transformant.

-----
Sources :
[1] glossary.html
[2] errors.html
[3] controlflow.html
[4] errors.html

>
Au revoir.
```

7.2 Vérification dans LangSmith

Chaque appel à `chain.invoke(...)` produit une **trace distincte** dans LangSmith. C'est un super outil pour :

- Repérer les questions qui ont mal marché.
- Comparer plusieurs reformulations d'une même question.
- Annoter manuellement les bonnes/mauvaises réponses (*thumbs up/down*) pour évaluer ton RAG.

Récapitulatif

Ce qu'il faut retenir :

1. *Pattern REPL* : init coûteux **hors** de la boucle, traitement rapide **dans** la boucle.
2. Gérer la terminaison propre : `EOFError`, `KeyboardInterrupt`, et ligne vide.
3. Une *CLI interactive* est le meilleur banc de test pour explorer son corpus et identifier les angles morts.
4. La chaîne RAG reste inchangée quand on passe à une UI web — c'est l'interface qui change, pas le *Runnable*.

8 Prompt strict pour refuser le hors-contexte

Exercice bonus 3. Bloquer les hallucinations polies

Pose une question dont la réponse **n'est pas** dans les pages indexées (par exemple : « *Comment installer un package avec pip ?* », sachant que la doc indexée couvre tutoriel + glossaire, pas l'installation). Observe le comportement par défaut. Puis durcis le prompt système pour que le modèle refuse explicitement de répondre hors-contexte.

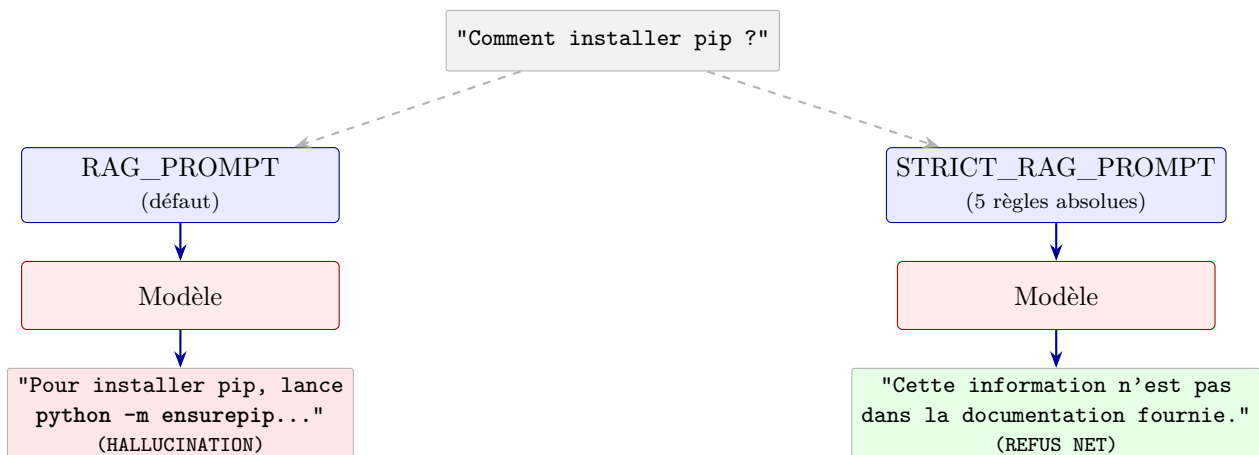
Contrainte : garder la même chaîne LCEL, juste swap le prompt. Le prompt strict doit **aussi** bien marcher sur des questions in-context (ne pas être excessivement restrictif).

Objectifs pédagogiques

- Reconnaître l'*hallucination polie* : le modèle complète avec ses connaissances générales quand le contexte ne suffit pas.
- Construire un prompt **défensif** avec règles numérotées et exemples de refus.
- Vérifier que le refus est exploitable en aval (router vers une autre source d'info, par exemple).

8.1 Solution

8.1.1 Architecture



8.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_core.output_parsers import StrOutputParser
3 from langchain_core.runnables import RunnablePassthrough
4 from config import get_model
5 from phase2_rag import format_docs
6
7
8 STRICT_RAG_PROMPT = ChatPromptTemplate.from_messages([
9     (
10         "system",
11         "Tu réponds EXCLUSIVEMENT à partir du contexte fourni ci-dessous. "
12         "Règles absolues :\n"
13         "1. Si la réponse n'apparaît pas littéralement dans le contexte, "
14         "tu réponds : 'Cette information n'est pas dans la documentation fournie.' "
15         "et tu t'arrêtes.\n"
16         "2. Tu ne complètes JAMAIS avec tes connaissances générales sur Python.\n"
17         "3. Tu ne fais JAMAIS d'inférence en partant d'un sujet voisin.\n"
18         "4. Si tu utilises le contexte, cite les chunks par leur numéro [N].\n"
19         "5. Si le contexte est ambigu ou partiel, dis explicitement ce qui "
20         "manque plutôt que d'extrapoler."
21     ),
22     (
23         "human",
24         "Contexte :\n{context}\n\nQuestion : {question}",
25     ),
26 ])
27
28
29 def build_strict_rag_chain(retriever):
30     return (
31         {
32             "context": retriever | format_docs,
33             "question": RunnablePassthrough(),
34         }
35         | STRICT_RAG_PROMPT
36         | get_model()
37         | StrOutputParser()
38     )
```

8.1.3 Explications pas à pas

Étape 1 — L'hallucination polie

Sans contraintes explicites, un LLM moderne complète volontiers ses réponses avec ses connaissances générales : c'est ce qu'il a appris à faire pour être « utile ». Sur une question hors-corpus, il va :

- Ignorer (ou paraphraser) le contexte non pertinent.
- Sortir une réponse plausible et fluide à partir de ses *connaissances pré-entraînées*.
- Ne pas mentionner que cette réponse ne vient PAS du contexte.

C'est l'*hallucination polie* : la réponse est techniquement correcte, mais elle vient d'une source non-traçable. En contexte pédagogique, c'est exactement ce qu'on veut éviter.

Étape 2 — Le pattern *règles absolues numérotées*

```
1 "Règles absolues :  
2 1. Si la réponse n'apparaît pas littéralement dans le contexte, ...  
3 2. Tu ne complètes JAMAIS avec tes connaissances générales sur Python.  
4 3. Tu ne fais JAMAIS d'inférence en partant d'un sujet voisin.  
5 4. Si tu utilises le contexte, cite les chunks par leur numéro [N].  
6 5. Si le contexte est ambigu ou partiel, dis explicitement ce qui manque."
```

Cinq caractéristiques d'un prompt défensif efficace :

- **Numérotation** : les modèles obéissent mieux à des règles listées qu'à un paragraphe.
- **Capitales** : EXCLUSIVEMENT, JAMAIS. Effet psychologique mesurable sur les LLM modernes.
- **Phrase exacte de refus** : « Cette information n'est pas dans la documentation fournie. » — le modèle a un *template* à recracher, pas à inventer.
- **Couverture explicite des cas voisins** : règle 3 anti- inférence (« cite-moi un sujet proche »), règle 5 anti-vague.
- **Garder la consigne positive** : règle 4 sur les citations, pour ne pas écraser le bon comportement par excès de restrictions.

Étape 3 — Vérifier la non-restriction excessive

Un piège classique des prompts stricts : ils deviennent *trop* restrictifs et refusent même des questions dont la réponse est dans le contexte. Test de contrôle indispensable :

```
1 in_context = "Qu'est-ce qu'un itérable en Python ?"           # dans le glossaire  
2 print(chain_strict.invoke(in_context))                         # doit répondre normalement
```

Si le prompt strict refuse cette question, c'est qu'il est mal calibré — il faut adoucir le ton sans perdre la rigueur sur le hors-contexte.

Étape 4 — Le refus exploitable

L'intérêt d'un refus net (toujours la même phrase) plutôt que d'un brouillon hallucinant : le *code applicatif* peut le détecter et router :

```
1 answer = chain_strict.invoke(question)  
2 if "Cette information n'est pas dans la documentation" in answer:  
3     # router vers une recherche web, un autre corpus, etc.  
4     fallback_chain.invoke(question)
```

*Un bon refus est **exploitable en aval**. L'hallucination polie ne l'est pas — il est impossible de savoir programmatiquement si la réponse est sourcée ou inventée. Toute architecture RAG sérieuse force le refus.*

Sortie de l'exécution

```
=====
EXERCICE 3 -- Prompt strict vs prompt par défaut
=====
[OK] Rechargement de l'index depuis ./chroma_db

Question (HORS contexte) : Comment installer le package `requests`
                           avec pip ?
```

```

--- Prompt PAR DEFAUT ---
Je ne peux pas répondre à cette question en m'appuyant sur le
contexte fourni.

Le contexte contient des informations sur le glossaire Python, le
chemin de recherche des modules (`sys.path`), les objets de type
chemin (`path-like object`), et les packages Python, mais il ne
contient aucune information sur l'installation de packages avec pip.

Pour répondre à ta question sur l'installation de `requests` avec
pip, il faudrait un contexte qui traite de la gestion des packages
et de l'outil pip, ce qui n'est pas disponible ici.

--- Prompt STRICT ---
Cette information n'est pas dans la documentation fournie.

Le contexte fourni traite de concepts comme les modules Python, le
chemin de recherche des modules (`sys.path`), les packages et les
PEPs, mais il ne contient aucune information sur l'installation de
packages avec pip.

Question (DANS contexte, contrôle) : Qu'est-ce qu'un itérable en
Python ?

--- Prompt STRICT ---
Selon la documentation fournie [2], un itérable est :

> Un objet capable de retourner ses membres un à un.

Les exemples d'itérables incluent :
- Tous les types de séquences (comme `list`, `str`, et `tuple`)
- Certains types non-séquences comme `dict` et les objets fichier
- Les objets de classes que vous définissez avec une méthode
  `__iter__()` ou une méthode `__getitem__()` qui implémente la
  sémantique de séquence

Les itérables peuvent être utilisés dans une boucle `for` et dans
de nombreux autres contextes où une séquence est nécessaire
(`zip()`, `map()`, etc.). Quand un objet itérable est passé à la
fonction intégrée `iter()`, elle retourne un itérateur pour cet
objet [2].

-----
OBSERVATIONS A RETENIR :
- Sans contrainte explicite, le modèle a tendance à 'aider' en
  complétant avec ses connaissances. C'est utile en chat général,
  c'est un BUG en RAG.
- Le prompt strict rend le refus net et exploitable en aval
  (tu peux router vers une autre source d'info, par exemple).

```

8.2 Vérification dans LangSmith

Lance la même question hors-contexte avec les deux prompts. Dans LangSmith :

1. Compare les *outputs* côte-à-côte : l'un parle de pip, l'autre refuse.
2. Compare les *system prompts* : le strict a 4-5 fois plus de tokens (pour mémoire, ~200 tokens supplémentaires).
3. Compare les *tokens de sortie* : le refus est court (~30 tokens), l'hallucination est longue (~300 tokens). Le prompt strict réduit la facture sur les requêtes hors-corpus.

Récapitulatif

Ce qu'il faut retenir :

1. L'hallucination polie est le piège n°1 du RAG : le modèle complète sans le dire avec ses connaissances générales.
2. Prompt défensif efficace : règles **numérotées**, **capitales** sur les interdictions, phrase exacte de refus, couverture des cas voisins.
3. Toujours tester avec une question **in-context** aussi, pour ne pas créer un prompt trop restrictif.
4. Un refus net (phrase canonique) est **exploitable en aval** : on peut router vers une autre source.
5. Coût : prompt système ~200 tokens en plus, mais réponse plus courte sur hors-contexte → facture similaire ou inférieure.

9 Vérifier et exploiter LangSmith

Exercice bonus 4. Activer le tracing et lire l'arbre d'exécution

Configure LangSmith dans ton `.env`, exécute une chaîne RAG, et explore l'arbre d'exécution dans l'UI `smith.langchain.com`. Vérifie programmatiquement la présence des variables d'environnement nécessaires.

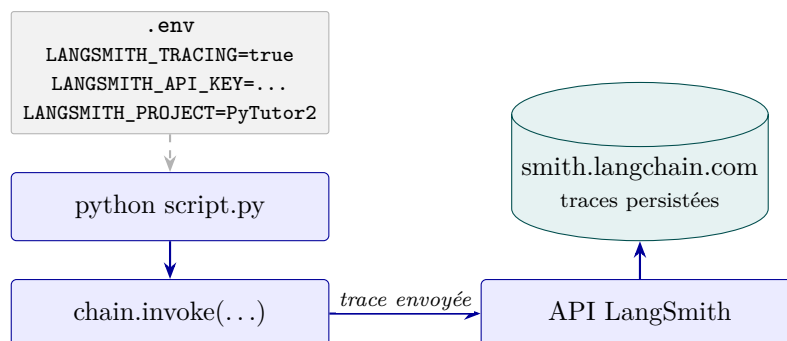
Contrainte : l'exercice doit afficher un message d'aide clair si la config est manquante (variables `LANGSMITH_TRACING`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`), avec les lignes à ajouter au `.env`.

Objectifs pédagogiques

- Configurer LangSmith : variables d'environnement et endpoint EU vs US.
- Lire l'arbre d'exécution d'une chaîne RAG : les branches du `RunnableParallel`, le `format_docs`, le prompt rendu, l'appel modèle, le parser.
- Apprendre à **utiliser LangSmith comme principale source de vérité** sur le comportement d'une chaîne (pas le code, pas les logs).

9.1 Solution

9.1.1 Architecture



9.1.2 Le code complet

```
1 import os
2 from phase2_rag import build_or_load_vectorstore, build_rag_chain_with_sources
3
4
5 def exercise_4_langsmith() -> None:
6     # --- Vérification de la configuration ---
```

```

7   tracing = os.getenv("LANGSMITH_TRACING", "").lower() in {"true", "1"}
8   api_key = os.getenv("LANGSMITH_API_KEY")
9   project = os.getenv("LANGSMITH_PROJECT", "default")
10
11  print(f"LANGSMITH_TRACING : {'V activ  ' if tracing else 'X NON activ  '}")
12  print(f"LANGSMITH_API_KEY : {'V pr  sente' if api_key else 'X absente'}")
13  print(f"LANGSMITH_PROJECT : {project}")
14
15  if not (tracing and api_key):
16      print("→ Ajoute dans .env :")
17      print("    LANGSMITH_TRACING=true")
18      print("    LANGSMITH_API_KEY=lsv2_pt_...")
19      print("    LANGSMITH_PROJECT=PyTutor2")
20      print("    LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com # si compte
    → EU")
21      return
22
23  # --- Ex  cution d'une cha  ne pour g  n  rer une trace ---
24  vectorstore = build_or_load_vectorstore()
25  retriever    = vectorstore.as_retriever(search_kwargs={"k": 4})
26  chain        = build_rag_chain_with_sources(retriever)
27
28  result = chain.invoke("Comment   crire un g  n  rateur infini en Python ?")
29  print(result["answer"][:300] + "...")

```

9.1.3 Explications pas    pas

  tape 1 — Les 3 variables d'environnement essentielles

| Variable | R  le |
|--------------------|--|
| LANGSMITH_TRACING | "true" pour activer l'envoi automatique des traces. Sans   a, les cha  nes s'ex  cutent normalement mais rien n'est envoy  . |
| LANGSMITH_API_KEY | Cl   personnelle (commence par lsv2_pt...).    cr  er sur smith.langchain.com (gratuit). |
| LANGSMITH_PROJECT | Nom du projet o   les traces seront group  es. Ex. : PyTutor2. |
| LANGSMITH_ENDPOINT | Optionnel. https://eu.api.smith.langchain.com pour un compte EU (d  faut : US). |

  tape 2 — V  rification programmatique

Lire `os.getenv` avec un d  faut explicite, et tester :

```

1 tracing = os.getenv("LANGSMITH_TRACING", "").lower() in {"true", "1"}
2 api_key = os.getenv("LANGSMITH_API_KEY")

```

Avantage : l'utilisateur voit *exactement* ce qui manque et a les lignes    coller dans son `.env`.

  tape 3 — L'arbre dans l'UI

Sur smith.langchain.com, ouvre ton projet et clique sur ta trace la plus r  cente. Le panneau gauche montre l'arbre d'ex  cution avec, pour la cha  ne RAG :

1. Le `RunnableParallel` (les 2 branches `context` et `question`).
2. Le `Retriever` (avec sa query exacte et la liste de docs).

3. Le `format_docs` (un `RunnableLambda`).
4. Le `ChatPromptTemplate` (avec le prompt rendu, texte complet envoyé au LLM).
5. L'appel modèle (avec tokens consommés, latence, coût \$).
6. Le `StrOutputParser`.

Étape 4 — Pourquoi LangSmith est plus utile que les logs Python

| Logs Python (<code>print</code> , <code>logging</code>) | LangSmith |
|---|--|
| Manuel : tu choisis quoi logger. | Automatique : tracé pour chaque <code>Runnable</code> . |
| Texte brut, dispersé sur stdout/fichier. | UI structurée, recherchable, partageable. |
| Pas de versionnage ni d'historique. | Toutes les traces archivées avec timestamp. |
| Pas de coût/latence agrégés. | Onglets dédiés <i>Costs</i> , <i>Latency</i> , <i>Tokens</i> . |
| Tu vois ce que TU as choisi de logger. | Tu vois exactement ce que le modèle a vu . |

*LangSmith devient la principale source de vérité dès qu'on travaille sur une chaîne sérieuse. Les **print** restent utiles pour le local quick-and-dirty, mais la moindre investigation passe par l'UI.*

Sortie de l'exécution

```
=====
EXERCICE 4 -- Tracer une exécution dans LangSmith
=====

LANGSMITH_TRACING : [OK] activé
LANGSMITH_API_KEY : [OK] présente
LANGSMITH_PROJECT : PyTutor2

Exécution d'une chaîne RAG (la trace sera envoyée à LangSmith)...
[OK] Rechargement de l'index depuis ./chroma_db

Réponse :
# Comment écrire un générateur infini en Python ?

Je dois te dire que **la réponse à cette question n'est pas
présente dans le contexte fourni**.

Le contexte explique comment fonctionnent les générateurs en
général [3,4] : ce sont des fonctions qui utilisent `yield` pour
retourner des valeurs une...

-----
CE QU'IL FAUT REGARDER DANS L'UI :
1. Va sur https://smith.langchain.com et ouvre le projet 'PyTutor2'
2. Tu vois ta dernière trace. Clique dessus.
3. Le panneau de gauche montre l'arbre d'exécution :
   - le RunnableParallel 'context' + 'question' (les 2 branches
     partent en parallèle)
   - puis le format_docs (RunnableLambda)
   - puis le prompt formaté (regarde le texte exact envoyé)
   - puis l'appel au modèle (avec coût et latence)
   - puis le parser
4. Clique sur l'appel modèle pour voir les tokens consommés
   et le prompt exact, sans rien deviner.
```

9.2 Vérification dans LangSmith

(Cet exercice EST la vérification.)

À regarder dans l'UI :

- L'onglet *Runs* liste toutes les traces récentes.
- Le panneau de *détail* d'une trace montre l'arbre.
- L'onglet *Costs* montre le coût \$ et les tokens.
- L'onglet *Annotations* permet de noter manuellement les bonnes/mauvaises réponses (*thumbs up/down*).

Récapitulatif

Ce qu'il faut retenir :

1. 3 variables d'env essentielles : `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY=lsv2_pt_...`, `LANGSMITH_PROJECT=ton-projet`.
2. Compte EU : ajouter `LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com`.
3. Vérification programmatique au démarrage de l'app : meilleure UX que de découvrir l'absence de trace plus tard.
4. L'UI LangSmith devient la principale source de vérité : arbre d'exécution, prompts rendus, coût, latence.
5. Annoter manuellement avec thumbs up/down crée un dataset d'évaluation gratuit.

10 MultiQueryRetriever : reformuler la question

Exercice bonus 5. Améliorer le recall avec un LLM en amont

Remplace le `Retriever` simple par un `MultiQueryRetriever` qui reformule chaque question utilisateur en 3 variantes via un LLM **avant** la recherche, puis fusionne les résultats des 3 recherches. Compare sur une question volontairement courte/ambiguë (ex. : « *Comment marche l'héritage ?* »).

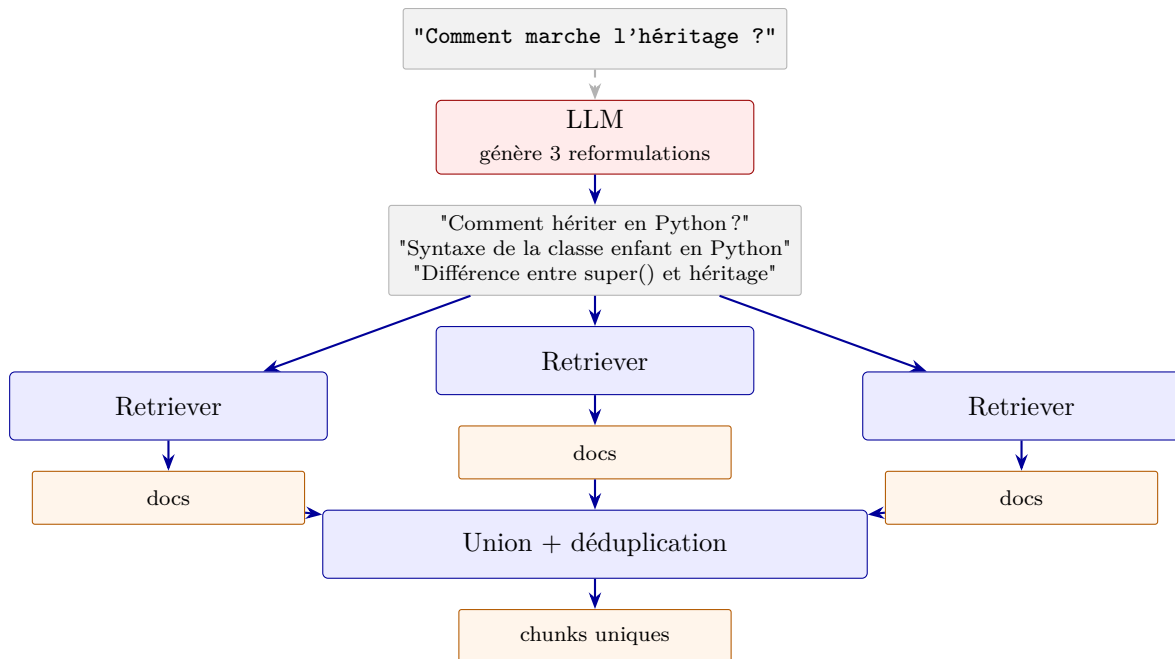
Contrainte : activer le logging du module `langchain.retrievers.multi_query` pour voir les reformulations générées par le LLM dans le terminal.

Objectifs pédagogiques

- Comprendre la notion de *recall* en information retrieval (retrouver toutes les bonnes infos, pas juste les plus pertinentes en surface).
- Utiliser un LLM pour **augmenter** la question utilisateur (et non pour répondre).
- Mesurer le coût supplémentaire (1 appel LLM de reformulation + N recherches au lieu d'une).
- Savoir quand utiliser MultiQuery : questions courtes / ambiguës ; et quand ne PAS l'utiliser : questions précises et bien formulées.

10.1 Solution

10.1.1 Architecture



Pipeline en quatre temps : reformulation LLM, N recherches parallèles, union, déduplication.

10.1.2 Le code complet

```
1 import logging
2 from langchain_classic.retrievers.multi_query import MultiQueryRetriever
3 from config import get_model
4 from phase2_rag import build_or_load_vectorstore, build_rag_chain
5
6
7 def exercise_5() -> None:
8     vectorstore = build_or_load_vectorstore()
9     base_retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
10    multi_retriever = MultiQueryRetriever.from_llm(
11        retriever=base_retriever,
12        llm=get_model(),
13    )
14
15    # Question courte et ambiguë : "héritage" peut couvrir plusieurs angles
16    question = "Comment marche l'héritage ?"
17
18    # --- Active le logging pour voir les reformulations ---
19    logging.basicConfig()
20    logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)
21
22    # --- Retriever simple ---
23    docs_simple = base_retriever.invoke(question)
24    print(f"--- Simple (k=3) ---")
25    for i, d in enumerate(docs_simple, 1):
26        print(f"  [{i}] {d.page_content[:100]}...")
27
28    # --- MultiQuery (les logs montrent les 3 reformulations) ---
29    docs_multi = multi_retriever.invoke(question)
30    print(f"\n--- MultiQuery ({len(docs_multi)} docs après déduplication) ---")
```

```

31 for i, d in enumerate(docs_multi, 1):
32     print(f"  [{i}] {d.page_content[:100]}...")
33
34 # --- Comparaison des réponses finales ---
35 print("\n--- RÉPONSE simple ---")
36 print(build_rag_chain(base_retriever).invoke(question)[:400], "...")
37
38 print("\n--- RÉPONSE MultiQuery ---")
39 print(build_rag_chain(multi_retriever).invoke(question)[:400], "...")

```

10.1.3 Explications pas à pas

Étape 1 — *Precision vs Recall*

| Métrique | Définition (en RAG) |
|------------------|---|
| Precision | Parmi les chunks retournés, combien sont pertinents ? |
| Recall | Parmi les chunks pertinents qui existent dans le corpus, combien sont retrouvés ? |

Le retriever simple optimise la *precision* (top-k par similarité exacte). **MultiQuery** optimise le *recall* : en multipliant les formulations de la question, on **couvre plus d'angles** et on retrouve des chunks qui étaient invisibles à la formulation originale.

Étape 2 — Comment la reformulation marche

`MultiQueryRetriever.from_llm` envoie un prompt au LLM :

« Génère 3 reformulations différentes de la question suivante pour améliorer une recherche par similarité : "Comment marche l'héritage ?" »

Le LLM produit (par exemple) :

- « Comment hériter d'une classe en Python ? »
- « Quelle est la syntaxe d'une classe enfant qui hérite ? »
- « Différence entre héritage et composition en Python »

Chaque variante est ensuite passée au retriever de base. Les résultats sont **fusionnés et dédupliques** (par identité de chunk).

Étape 3 — Le logging interne

```

1 import logging
2 logging.basicConfig()
3 logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)

```

Avec ces 3 lignes, les reformulations générées s'affichent dans la console à chaque appel. Indispensable pour :

- Comprendre ce que le LLM *reformule*.
- Diagnostiquer si les reformulations sont pertinentes ou triviales.
- Ajuster le **prompt** de reformulation au besoin.

Étape 4 — Coût et quand l'utiliser

| Métrique | Simple | MultiQuery |
|-------------------------------------|--------------------|---|
| Appels LLM pour la query | 0 (embedding seul) | 1 (reformulation) |
| Appels au retriever | 1 | N (typique : 3) |
| Coût par requête utilisateur | ~1x | ~1.5–2x |
| Gain sur questions précises | — | Nul , juste du coût |
| Gain sur questions courtes/ambiguës | — | Significatif (+30–50% de recall) |

***Règle pratique** : utilise MultiQuery quand tu observes que des réponses sont incomplètes parce que des chunks pertinents n'ont pas été ramenés. N'utilise pas MultiQuery si le problème est ailleurs (chunk_size, prompt, modèle).*

Sortie de l'exécution

```
[OK] Rechargement de l'index depuis ./chroma_db

Question (volontairement vague) : Comment marche l'héritage ?

--- Retriever SIMPLE (k=3) ---
[1] 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning',
    'UserWarning', 'ValueError', 'Warning', '...'
[2] |     raise ExceptionGroup('there were problems', excs) |
    ExceptionGroup: there were problems (2 sub...)
[3] definition. (A class is never used as a global scope.) While
    one rarely encounters a good reason f...

--- MultiQuery Retriever ---

11 documents au total (déduplication automatique)
[1] 4.9. More on Defining Functions 4.9.1. Default Argument Values
    4.9.2. Keyword Arguments 4.9.3. Speci...
[2] Previous topic Enum HOWTO Next topic Logging HOWTO This page
    Report a bug Improve this page Show sou...
[3] mixed numeric types are compared according to their numeric
    value, so 0 equals 0.0, etc. Otherwise,...
[4] . text encoding A string in Python is a sequence of Unicode
    code points (in range U+0000 - U+10FFF...
[5] Encoding and error handler used by Python to decode bytes from
    the operating system and encode Unico...
[6] simply replace the base class method of the same name. There is
    a simple way to call the base class ...
[7] There's nothing special about instantiation of derived classes:
    DerivedClassName() creates a new ins...
[8] |     raise ExceptionGroup('there were problems', excs) |
    ExceptionGroup: there were problems (2 sub...)
[9] cycles so that the memory can be reclaimed. data race A
    situation where multiple threads access th...
[10] yield and a return statement is that on reaching a yield the
    generator's state of execution is suspe...
[11] __iter__() method which returns an object with a __next__()
    method. If the class defines __next__()...

--- REPONSE avec retriever simple ---
# L'héritage en Python

D'après le contexte fourni [3], voici comment fonctionne l'héritage :

## Syntaxe de base
```

Pour créer une classe qui hérite d'une autre, on utilise la syntaxe suivante :

```
```python
class DerivedClassName(BaseClassName):
 <statement-1>
 ...
 <statement-N>
```

**Explication :**
- `DerivedClassName` est la classe dérivée (la classe enfant)
- `BaseClassName` est la cl...

--- REPONSE avec MultiQuery ---
# L'héritage en Python

D'après la documentation fournie [6][7], voici comment fonctionne l'héritage :

## Syntaxe de base

Une classe dérivée hérite d'une ou plusieurs classes de base :

```python
class ClasseDerivee(ClasseBase):
 <instruction-1>
 ...
 <instruction-N>
```

## Fonctionnement principal

**Résolution des méthodes** [7] : Quand vous appelez une méthode, Python cherche l'attr...

-----
OBSERVATIONS A RETENIR :
- Regarde les logs ci-dessus pour voir les 3 reformulations générées automatiquement par le LLM.
- MultiQuery double ou triple ton coût par question (1 appel pour reformuler + N appels de recherche), mais améliore notablement le recall sur les questions courtes ou floues.
- A ne PAS utiliser sur des questions très précises et bien formulées : aucun gain, juste du coût en plus.
```

10.2 Vérification dans LangSmith

Avec MultiQuery, LangSmith trace un arbre plus profond :

```
MultiQueryRetriever
├── LLMChain (génération des 3 reformulations) ─ LLM #1
│   ├── VectorStoreRetriever (variante 1)
│   ├── VectorStoreRetriever (variante 2)
│   └── VectorStoreRetriever (variante 3)
│       └── (fusion + déduplication, hors trace)
```

*Tu vois directement les 3 variantes générées (onglet Output de la **LLMChain**), et le contenu des 3 retrievals. Précieux pour auditer la qualité des reformulations.*

Récapitulatif

Ce qu'il faut retenir :

1. *MultiQueryRetriever* reformule la question en N variantes **via un LLM**, puis fusionne les N résultats du retriever de base.
2. Objectif : améliorer le recall sur les questions courtes, ambiguës ou mal formulées.
3. Coût : +1 appel LLM pour reformuler + N appels au retriever. ~ 1.5 à $2\times$ le coût d'une requête simple.
4. Active le `logging` sur `langchain.retrievers.multi_query` pour voir les reformulations.
5. À ne PAS utiliser sur des questions déjà précises : aucun gain, juste du coût en plus.

11 Filtrage par métadonnée

Exercice bonus 6. Restreindre la recherche à une sous-partie du corpus

Chaque chunk du Chroma a hérité d'une `metadata.source` (l'URL d'origine). Construis un retriever filtré qui ne cherche que dans `tutorial/classes.html` pour une question sur la POO. Compare avec le retriever non filtré.

Démontre aussi le **piège** : si tu poses une question hors du périmètre du filtre (ex. : « Comment lever une exception ? » alors que le filtre est sur `classes.html`), le retriever ramène du contenu hors-sujet.

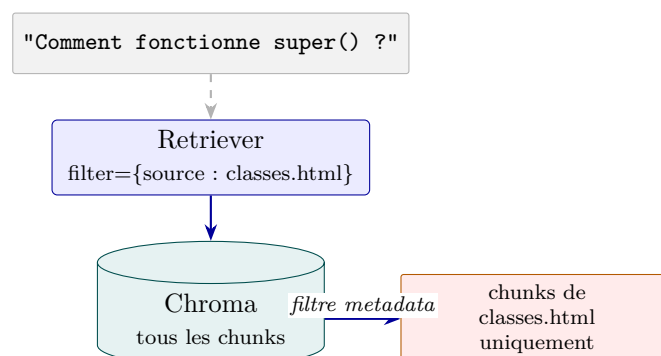
Contrainte : utiliser `search_kwargs={"filter": {...}}` avec la syntaxe MongoDB-like de Chroma.

Objectifs pédagogiques

- Utiliser la `metadata` des chunks pour restreindre la recherche.
- Connaître la syntaxe de filtre Chroma (`$eq`, `$in`, `$and`, `$or`, ...).
- Identifier le piège : un filtre mal calibré force le retriever à choisir le « moins pire » dans un sous-corpus inadapté.
- Esquisser le *pattern de production* : un classifieur LLM en amont qui décide *dynamiquement* du filtre à appliquer.

11.1 Solution

11.1.1 Architecture



Le filtre s'applique *avant* le calcul de similarité : seuls les chunks dont les metadata matchent sont éligibles à la recherche.

11.1.2 Le code complet

```
1 from phase2_rag import build_or_load_vectorstore
2
3
4 def exercise_6() -> None:
5     vectorstore = build_or_load_vectorstore()
6
7     # --- Retriever filtré : SEULEMENT classes.html ---
8     classes_only = vectorstore.as_retriever(
9         search_kwargs={
10             "k": 4,
11             "filter": {"source": "https://docs.python.org/3/tutorial/classes.html"},
12         },
13     )
14
15     # --- Retriever non filtré pour comparaison ---
16     unfiltered = vectorstore.as_retriever(search_kwargs={"k": 4})
17
18     question = "Comment fonctionne `super()` ?"
19
20     print("--- SANS filtre ---")
21     for i, d in enumerate(unfiltered.invoke(question), 1):
22         src = d.metadata.get("source", "?").split("/")[-1]
23         print(f" [{i}] ({src}) {d.page_content[:80]}...")
24
25     print("\n--- AVEC filtre source=classes.html ---")
26     for i, d in enumerate(classes_only.invoke(question), 1):
27         src = d.metadata.get("source", "?").split("/")[-1]
28         print(f" [{i}] ({src}) {d.page_content[:80]}...")
29
30     # Démo de la limite : question hors du périmètre du filtre
31     off_topic = "Comment lever une exception ?"
32     print(f"\n--- AVEC filtre, question hors-périmètre ({off_topic}) ---")
33     for i, d in enumerate(classes_only.invoke(off_topic), 1):
34         src = d.metadata.get("source", "?").split("/")[-1]
35         print(f" [{i}] ({src}) {d.page_content[:80]}...")
```

11.1.3 Explications pas à pas

Étape 1 — La syntaxe Chroma

```
1 search_kwargs={
2     "k": 4,
3     "filter": {"source": "https://docs.python.org/3/tutorial/classes.html"},
4 }
```

Forme simple : {key: value} = match exact sur la metadata. Sous le capot, Chroma traduit en {key: {\$eq: value}}.

Pour des filtres plus riches, syntaxe MongoDB-like :

| Opérateur | Exemple |
|--------------------------|------------------------------------|
| \$eq, \$ne | {"source": {"\$eq": "url"}} |
| \$in, \$nin | {"source": {"\$in": [url1, url2]}} |
| \$gt, \$gte, \$lt, \$lte | sur des champs numériques |
| \$and, \$or | {"\$and": [filtre1, filtre2]} |

Étape 2 — Quand le filtre brille

Cas d'usage parfaits :

- **Multi-tenant** : chaque utilisateur a son propre corpus, filtrer par `tenant_id`.
- **Multi-langue** : filtrer par `lang` pour éviter de retrouver du contenu en anglais sur une query française.
- **Documentation versionnée** : filtrer par `version` (« `super()` en Python 3.0 vs 3.12 »).
- **Dates** : récupérer uniquement les chunks plus récents que X.

Étape 3 — Quand le filtre casse

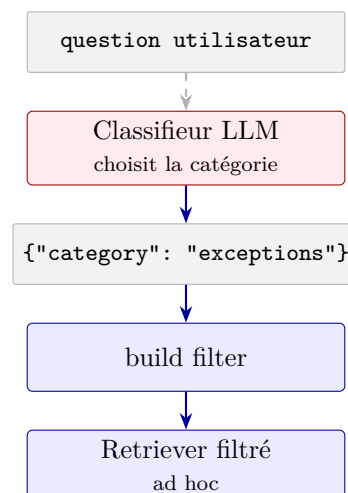
Si le filtre est **mal calibré** (utilisateur cherche dans le mauvais sous-corpus), le retriever ramène du contenu hors-sujet sans warning :

Sur « Comment lever une exception ? » avec `filter classes.html`, Chroma retourne les chunks de `classes.html` les plus proches de la query — qui parlent de classes, pas d'exceptions. Le RAG aval va générer une réponse hors-sujet ou hallucinée à partir de ce contexte inadapté.

C'est le danger du filtre : il **force** le retrieval à choisir dans un sous-ensemble, même si rien n'y est pertinent.

Étape 4 — Pattern de production : auto-routing

En production, le filtre ne devrait pas être codé en dur. Pattern typique :



Le mini-classifieur LLM (avec `with_structured_output + Literal[...]` comme dans la Phase 1 / Exercice 4 bonus) décide *dynamiquement* du filtre à appliquer. C'est l'analogue du `RunnableBranch` mais appliqué au retrieval.

Sortie de l'exécution

```
=====
EXERCICE 6 -- Filtrage par métadonnée (source URL)
=====

[OK] Rechargement de l'index depuis ./chroma_db

Question : Comment fonctionne `super()` ?

--- SANS filtre (toutes les pages indexées) ---
[1] (modules.html) 'UnicodeError', 'UnicodeTranslateError',
    'UnicodeWarning', 'UserWarning', 'Value...
[2] (modules.html) 'EOFError', 'Ellipsis', 'EnvironmentError',
    'Exception', 'False', 'FileExistsErr...
[3] (modules.html) Function    The built-in function dir() is used
    to find out which names a module ...
[4] (glossary.html) . single dispatch  A form of generic function
    dispatch where the implementation...

--- AVEC filtre source=classes.html ---
[1] (classes.html) definition. (A class is never used as a global
    scope.) While one rarely encoun...
[2] (classes.html) call-next-method and is more powerful than the
    super call found in single-inheri...
[3] (classes.html) __iter__() method which returns an object with
    a __next__() method. If the clas...
[4] (classes.html) # Function defined outside the class def f1 (
    self , x , y ): return min ( x , x...

Démon limite -- question hors-périmètre : Comment lever une exception ?

--- AVEC filtre source=classes.html (mauvaise idée ici) ---
[1] (classes.html) 285 >>> xvec = [ 10 , 20 , 30 ] >>> yvec = [ 7 ,
    5 , 3 ] >>> sum ( x * y for x ,...
[2] (classes.html) 9. Classes 9.1. A Word About Names and Objects
    9.2. Python Scopes and Namespaces...
[3] (classes.html) Examples, recipes, and other code in the
    documentation are additionally licensed...
[4] (classes.html) __iter__() method which returns an object with
    a __next__() method. If the clas...

-----
OBSERVATIONS A RETENIR :
- Le filtre par metadata est un couteau : utile quand tu SAIS
  où chercher (ex: namespace, tag, langue, date), pénalisant
  quand tu te trompes de domaine.
- Syntaxe Chroma : opérateurs $eq, $ne, $in, $gt, $gte, $lt,
  $lte, $and, $or. Ex: {'source': {'$in': [url1, url2]}}.
- Pattern de prod : un mini-classifieur LLM en amont qui décide
  du filtre à appliquer (auto-routing par metadata).
```

11.2 Vérification dans LangSmith

Le run du retriever filtré expose le `filter` dans son input :

```
VectorStoreRetriever (filtered)
Input : "Comment fonctionne super() ?"
Config : k=4, filter={source: ".../classes.html"}
Output : [Doc, Doc, Doc, Doc]
tous source: classes.html
```

Vérifie toujours dans LangSmith que la `metadata.source` des chunks retournés est bien dans le périmètre attendu. Si tu vois des sources hors du filtre, c'est qu'il y a un bug dans la spécification (souvent une typo dans l'URL).

Récapitulatif

Ce qu'il faut retenir :

1. `search_kwargs={"filter": {...}}` restreint le retrieval aux chunks dont la `metadata` matche.
2. Syntaxe Chroma MongoDB-like : `$eq`, `$in`, `$and`, etc. Match exact si on omet l'opérateur.
3. Bons cas d'usage : multi-tenant, multi-langue, versionnage, dates.
4. Piège : si la question sort du périmètre du filtre, le retriever ramène du contenu hors-sujet sans warning. Toujours vérifier.
5. Pattern de production : classifieur LLM en amont qui décide dynamiquement du filtre à appliquer (auto-routing par metadata).

A Programme principal exécutable

La fonction `demo()` ci-dessous orchestre les trois démos présentées tout au long du chapitre : indexation (ou rechargement), RAG simple, RAG avec sources, puis comparaison `similarity` vs `mmr`. C'est le point d'entrée exécuté par `python phase2_rag.py`.

```
1 def demo() -> None:
2     # --- Indexation (ou rechargement) ---
3     vectorstore = build_or_load_vectorstore()
4
5     # --- Démo 1 : RAG simple, sortie texte ---
6     print("=" * 70)
7     print("DÉMO 1 - RAG simple (retriever similarity, k=4)")
8     print("=" * 70)
9     retriever = vectorstore.as_retriever(
10         search_type="similarity",
11         search_kwargs={"k": 4},
12     )
13     chain = build_rag_chain(retriever)
14
15     question = DEMO_QUESTIONS[1]
16     print(f"\nQuestion : {question}\n")
17     answer = chain.invoke(question)
18     print(answer)
19
20     # --- Démo 2 : RAG avec sources ---
21     print("\n" + "=" * 70)
22     print("DÉMO 2 - RAG avec inspection des sources")
23     print("=" * 70)
24     chain_with_sources = build_rag_chain_with_sources(retriever)
25     question = DEMO_QUESTIONS[2]
26     print(f"\nQuestion : {question}\n")
27     result = chain_with_sources.invoke(question)
28     print("RÉPONSE :")
29     print(result["answer"])
30     print("\nSOURCES UTILISÉES :")
31     for i, doc in enumerate(result["context"], 1):
32         snippet = doc.page_content[:120].replace("\n", " ")
33         print(f"  [{i}] {doc.metadata.get('source', '?')}")
34         print(f"      {snippet}...")
35
```

```

36 # --- Démo 3 : comparaison similarity vs MMR ---
37 print("\n" + "=" * 70)
38 print("DÉMO 3 - Similarity vs MMR sur la MÊME question")
39 print("=" * 70)
40 question = DEMO_QUESTIONS[0]
41 print(f"\nQuestion : {question}")
42
43 sim_retriever = vectorstore.as_retriever(
44     search_type="similarity",
45     search_kwargs={"k": 3},
46 )
47 mmr_retriever = vectorstore.as_retriever(
48     search_type="mmr",
49     # fetch_k : on récupère 20 candidats avant de sélectionner
50     # les 3 plus diversifiés.
51     # lambda_mult : 1.0 = pure similarité, 0.0 = pure diversité.
52     search_kwargs={"k": 3, "fetch_k": 20, "lambda_mult": 0.5},
53 )
54
55 print("\n--- Top-3 similarity ---")
56 for i, doc in enumerate(sim_retriever.invoke(question), 1):
57     print(f"  [{i}] {doc.page_content[:100].replace(chr(10), ' ')}...")
58
59 print("\n--- Top-3 MMR (diversifié) ---")
60 for i, doc in enumerate(mmr_retriever.invoke(question), 1):
61     print(f"  [{i}] {doc.page_content[:100].replace(chr(10), ' ')}...")
62
63 print("\n" + "=" * 70)
64 print("Observe : MMR évite les doublons sémantiques que retourne la")
65 print("similarity pure. Pour une question généraliste comme 'liste vs'")
66 print("tuple', MMR ramène souvent des chunks plus complémentaires.")
67 print("=" * 70)
68
69 if __name__ == "__main__":
70     demo()

```